



**POLITÉCNICO
DE LEIRIA**

ESCOLA SUPERIOR
DE TECNOLOGIA
E GESTÃO

Plot In a Pot

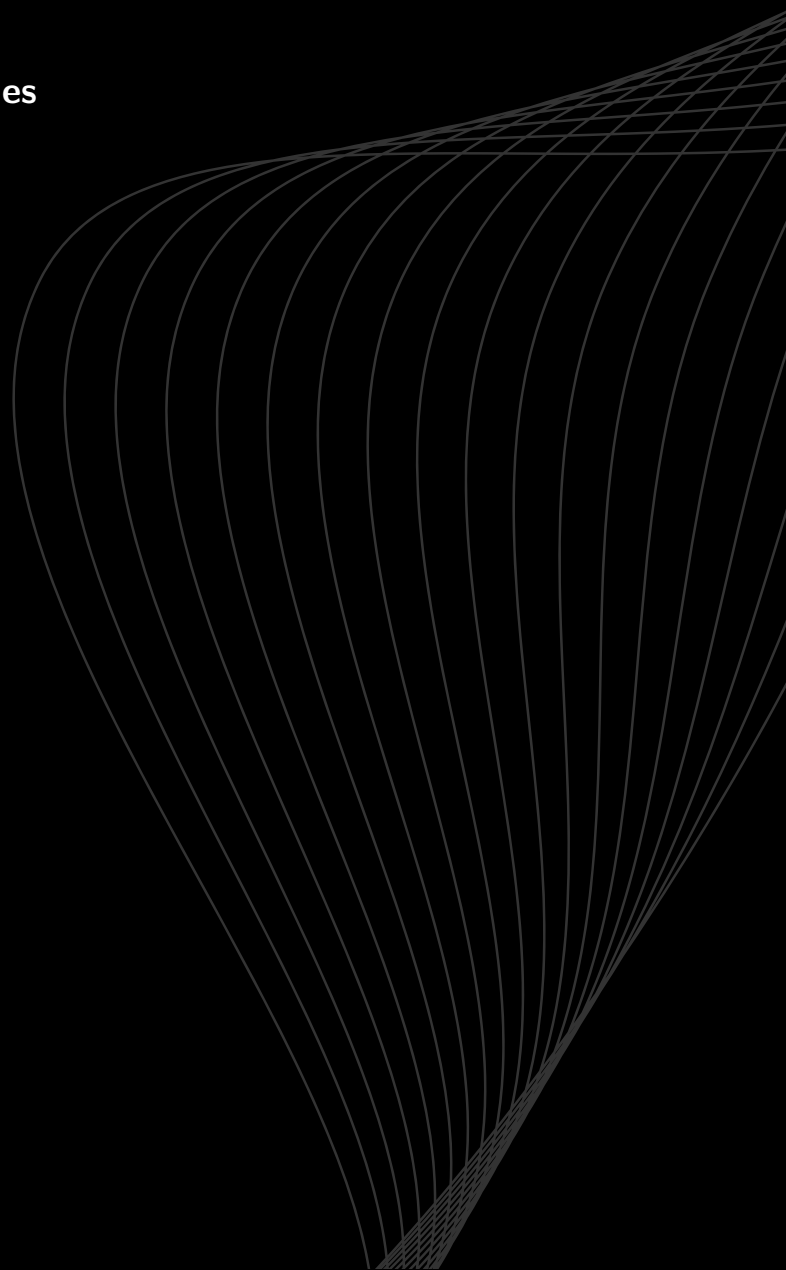
Uma Ferramenta para Narrativas Não Lineares

Miguel Gomes Silva

Tomás Alexandre dos Santos Reis Lopes

Escola Superior de Tecnologia e Gestão
Licenciatura em Engenharia Informática
UC de Projeto Informático

Leiria, Julho 2026



Plot In a Pot

Uma Ferramenta para Narrativas Não Lineares

Miguel Gomes Silva

Student No. 2221462

Tomás Alexandre dos Santos Reis Lopes

Student No. 2222970

Supervisores: Anabela Gonçalves Rodrigues Marto

*Professora Adjunta, Escola Superior de
Tecnologia e Gestão do Politécnico de Leiria*

Miguel Cerdeira Marreiros Negrão

*Professor Adjunto, Escola Superior de
Tecnologia e Gestão do Politécnico de Leiria*

Nelson Martins Ferreira

*Professor Adjunto, Escola Superior de
Tecnologia e Gestão do Politécnico de Leiria*

Escola Superior de Tecnologia e Gestão

Departamento de Engenharia Informática

Licenciatura em Engenharia Informática

UC de Projeto Informático

Trabalho de Projeto da unidade curricular de Projeto Informático realizado sob a orientação da Professora Doutora Anabela Gonçalves Rodrigues Marto e do Professor Doutor Miguel Cerdeira Marreiros Negrão e do Professor Doutor Nelson Martins Ferreira

Leiria, Julho 2026

Plot In a Pot

Copyright © 2026 - Miguel Gomes Silva e Tomás Alexandre dos Santos Reis Lopes,
Escola Superior de Tecnologia e Gestão.

O presente relatório é um trabalho original, elaborado exclusivamente para este fim, tendo sido devidamente citados todos os autores cujos estudos contribuíram para a sua elaboração. É permitida a sua reprodução parcial com indicação dos autores e referência ao grau, ano letivo, instituição—*Politécnico de Leiria*—e data da defesa pública.

O presente trabalho beneficiou da utilização do modelo *IPLeiria-Thesis*.

Agradecimentos

Gostaríamos de expressar o nosso mais sincero agradecimento a todos aqueles que, direta ou indiretamente, contribuíram para a realização deste projeto informático e para a elaboração deste relatório.

À nossa orientadora, Professora Doutora Anabela Gonçalves Rodrigues Marto, pelo voto de confiança, paciência e precioso apoio técnico e científico, essenciais para superar os desafios metodológicos e organizacionais deste trabalho. Estendemos os nossos agradecimentos aos coorientadores, Professor Doutor Nelson Martins Ferreira e Professor Doutor Miguel Cerdeira Marreiros Negrão. O rigor do Professor Nelson na abordagem matemática e a experiência do Professor Miguel na arquitetura de sistemas informáticos trouxeram discussões valiosas e revisões que possibilitaram elevar a qualidade técnica da nossa solução.

À Escola Superior de Tecnologia e Gestão (ESTG) do Politécnico de Leiria, por disponibilizar os recursos académicos e o ambiente propício ao nosso desenvolvimento pessoal e profissional ao longo da Licenciatura em Engenharia Informática.

Aos participantes que voluntariamente colaboraram nos testes de usabilidade e acessibilidade, cujo feedback construtivo e sincero foi inestimável para a validação e refinamento prático do software.

Por fim, expressamos um agradecimento muito especial às nossas famílias e amigos, pelo apoio incondicional, incentivo constante e paciência demonstrados durante este percurso académico.

Resumo

O desenvolvimento de narrativas interativas e jogos baseados em escolhas enfrenta sérios desafios de usabilidade e integridade estrutural à medida que a ramificação narrativa cresce. Embora existam ferramentas populares de autoria como o Twine, a maioria carece de suporte nativo integrado para gestão de localização multi-idioma e de motores de validação estática de caminhos lógicos para detetar inconsistências (como becos sem saída e cenas órfãs) antes da fase de compilação ou execução. O presente trabalho de projeto propõe o *Plot in a Pot* (PiaP), um editor visual e simulador baseado em grafos que unifica a autoria de narrativas escritas no formato aberto Twee 3 (SugarCube) com um motor integrado de tradução de textos e validação estática de caminhos.

Implementado em React e no ecossistema React Flow, o PiaP destaca-se pela sua estética neo-brutalista de alto contraste visual, concebida para mitigar barreiras de acessibilidade visual e cromática para autores com daltonismo severo (como deuteranopia). O sistema integra: (i) uma matriz de localização baseada em grelha bidimensional com importação e exportação de CSV; (ii) um motor de validação estática suportado por algoritmos de Pesquisa em Largura (BFS) para analisar a acessibilidade de cenas; (iii) um modo de simulação interativo dotado de um agente inteligente autónomo (*Smart Walker*) que utiliza procura por novidade (*Novelty Search*) para percorrer e testar caminhos automaticamente; e (iv) a visualização de gráficos com arestas curvas para ligar as cenas.

A avaliação qualitativa e quantitativa do sistema foi efetuada através de testes com seis utilizadores com diferentes perfis técnicos e de acessibilidade. Os resultados revelaram uma taxa média de conclusão de tarefas de 91,7% e uma elevada usabilidade e satisfação, validando a eficácia do *Plot in a Pot* como uma ferramenta viável e robusta para autores e *game designers* na prototipagem e desenvolvimento de narrativas interativas complexas. Como trabalhos futuros, perspetiva-se a execução de IA local via WebGPU, edição colaborativa em tempo real (via modelos Ponto a ponto (Peer-to-peer) (P2P)) e model checking formal da lógica narrativa.

Palavras-Chave: Narrativa Interativa, Editores de Grafos, Localização de Jogos, Acessibilidade Visual, Validação Estática de Fluxo, Twine, SugarCube.

Abstract

The development of interactive narratives and choice-based games faces serious challenges in usability and structural integrity as the narrative branching expands. Although popular authoring tools like Twine exist, most lack integrated native support for multi-language localization management and static path-validation engines to detect inconsistencies (such as dead ends and orphan scenes) before compilation or execution. This project work proposes *Plot in a Pot* (PiaP), a graph-based visual editor and simulator that unifies the authoring of interactive stories written in the open Tweep 3 format (SugarCube) with an integrated translation matrix and static path-validation engine.

Implemented in React and using the React Flow ecosystem, PiaP stands out for its high-contrast neo-brutalist visual style, specifically designed to mitigate visual and chromatic accessibility barriers for authors with severe color blindness (such as deuteranopia and protanopia). The system features: (i) a translation matrix layout with CSV import and export capabilities; (ii) a static validation engine powered by Breadth-First Search (BFS) algorithms to audit scene reachability; (iii) an interactive simulation mode equipped with an autonomous agent (*Smart Walker*) that leverages novelty search heuristics to automatically test paths; and (iv) graph visualization with curved edges to connect scenes.

Qualitative and quantitative evaluation was conducted through user testing with six participants of varying technical backgrounds and accessibility needs. The results showed a 91.7% average task completion rate alongside high levels of usability and satisfaction, validating the effectiveness of *Plot in a Pot* as a viable and robust tool for authors and game designers during the prototyping and development of complex interactive narratives. Future work directions include local AI execution via WebGPU, real-time collaborative editing (via P2P models), and formal model checking of narrative logic.

Keywords: Interactive Narrative, Graph Editors, Game Localization, Visual Accessibility, Static Flow Validation, Twine, SugarCube.

Declaração sobre o Uso de Inteligência Artificial

i Exemplo de Utilização

Reconheço a utilização integrada da inteligência artificial Gemini (<https://gemini.google.com>) em conjunto com o ecossistema e agente autónomo de desenvolvimento Google Antigravity (<https://antigravity.google>) como ferramentas de suporte no processo de codificação, desenho arquitetural e automação de testes lógicos da aplicação *Plot in a Pot*. A inteligência artificial operou como assistente no espaço de trabalho local seguro (*sandbox*), executando tarefas de refatorização e análise de dependências sob estrita validação e supervisão humana.

Descrição da Utilização

Instrução 1: Analisa a estrutura de dados atual do interpretador visual e implementa uma rotina modular capaz de ler o formato de ficheiros de texto do motor de jogo, isolando a lógica de transição dos nós para evitar quebras no processamento da árvore narrativa.

Resposta 1: [O agente do Antigravity acedeu ao ambiente local através do ciclo de planeamento e execução, estruturando os módulos iniciais do interpretador de dados e aplicando funções de higienização de texto para garantir a integridade dos caminhos e ligações visuais.]

Instrução 2: Executa a análise de impacto estrutural no código da aplicação caso o serviço de backend mude o motor de inferência local assente em microsserviços externos para uma execução nativa baseada em computação gráfica no lado do cliente.

Resposta 2: [O ecossistema inspecionou as dependências do repositório, mapeou o fluxo de dados dos serviços de inteligência artificial e reestruturou os métodos para gerir o ciclo de vida do novo motor diretamente no navegador da Web, sugerindo melhorias na uniformização dos identificadores estruturais.]

Processo de Validação Humana e Decisão Prática: A utilização do ecossistema Antigravity e do Gemini serviu para acelerar o desenvolvimento de protótipos e mapear o impacto de alterações estruturais na aplicação. No entanto, todas as decisões finais de

engenharia de software, modelação definitiva da base de dados de localização e optimização dos algoritmos de validação do fluxo narrativo foram revistas, corrigidas e implementadas manualmente pelos autores. A arquitetura computacional final e a sua conformidade técnica permanecem sob o total controlo e responsabilidade intelectual da equipa de desenvolvimento humana.

Conteúdo

<i>Lista de Figuras</i>	xv
<i>Lista de Tabelas</i>	xvii
<i>Siglas</i>	xix
1 Introdução	1
1.1 Objeto do Trabalho	1
1.2 Justificação e Pertinência do Tema	1
1.3 Objetivos do Trabalho	2
1.3.1 Objetivo Geral	2
1.3.2 Objetivos Específicos	2
1.4 Estrutura do Documento	3
2 Fundamentação Teórica e Estado da Arte	4
2.1 Contexto Teórico	4
2.2 Conceitos Fundamentais de Narrativas Não-Lineares	5
2.3 Teoria de Grafos na Narrativa Interativa	6
2.3.1 Tipos de Grafos e Aplicação Narrativa	6
2.3.2 Modelação por Grafo Controlado por Estados	7
2.3.3 O Desafio da Explosão Combinatória	7
2.4 Software de Criação de Histórias não Lineares	9
2.4.1 Twine	9
2.4.2 Ink (Inkle Studios)	10
2.4.3 ChoiceScript	11
2.4.4 Yarn Spinner	11
2.4.5 Narrative Flow	12
2.4.6 Articy Draft	12
2.4.7 Análise Comparativa	13
2.5 O Formato Tweak3 e Linguagens de Scripting Narrativo	14
2.5.1 Sintaxe de Ligações e Transições	14
2.5.2 Metadados Geométricos e Etiquetas (Tags)	15
2.5.3 Destinos Dinâmicos vs. Validação Estática	15
2.5.4 Macros e Lógica Condicional	16
2.6 Integração de Modelos de Linguagem de Grande Escala (LLMs) Locais . .	16

2.6.1	Análise de Famílias de Modelos e Recomendações	16
3	Conceção e Desenvolvimento	18
3.1	Arquitetura Geral do Sistema	18
3.2	Requisitos do Sistema	19
3.2.1	Requisitos Funcionais	19
3.2.2	Requisitos Não Funcionais	20
3.3	Métodos, Técnicas e Enquadramento Tecnológico	21
3.3.1	Justificação das Decisões de Design	21
3.4	Modelação de Dados: Lista de Adjacência	22
3.5	Parser Síncrono e Especificação Twee 3	23
3.5.1	Importação e Conversão de Coordenadas de Zonas	23
3.5.2	Passagens Especiais	23
3.5.3	Modelo SugarCube (Ligações e Macros)	24
3.6	Interação Visual, Edição e Acessibilidade	24
3.6.1	Zonas de Agrupamento Visual	24
3.6.2	Arestas Curvadas com Waypoints	25
3.6.3	Acessibilidade e Identidade Visual Customizada	25
3.6.4	Modelação de Dados e Sincronização com o React Flow	26
3.7	Validação Estática e Simulação	28
3.7.1	Algoritmo de Validação Estática e Exploração do Espaço de Esta- dos (Pesquisa em Largura (Breadth-First Search) (BFS))	28
3.7.2	Simulador em Tempo Real (PlayMode) e Smart Walker	30
3.7.3	Motor de Compilação JIT de Macros SugarCube	30
3.8	Integração de Modelos de Linguagem (LLMs)	31
3.8.1	Arquitetura de Integração Local	31
3.8.2	Papel do LLM: Conversão e Importação Automática de Histórias	32
4	Implementação e Avaliação	33
4.1	Estrutura do Código e Detalhes de Implementação	33
4.1.1	Gestão de Estado Global, Reatividade e Histórico	33
4.1.2	Componentes Gráficos e Desafios de Interação (Canvas, Nós e Arestas)	35
4.1.3	Mapeamento Reverso, Expressões Regulares e Matriz de Localização	37
4.1.4	Sandbox de Simulação, Edição Lógica Estruturada e Utilitários	40
4.2	Testes de Conversão e Comparação de Modelos de Linguagem (IA Local)	45
4.2.1	Primeira Fase: Testes em Máquina Local (RTX 3050)	45
4.2.2	Segunda Fase: Testes com Maior Capacidade de Processamento	45
4.2.3	Identificação do Bug Técnico no Ollama	46
4.2.4	Estado Atual da Funcionalidade (Beta) e Trabalho Futuro	46
4.3	Análise de Desempenho e Complexidade Algorítmica	47
4.3.1	Complexidade Algorítmica Teórica	47

4.3.2	Benchmarks de Desempenho Empíricos	47
4.4	Metodologia dos Testes de Utilizador	48
4.4.1	Perfil dos Participantes	48
4.4.2	Tarefas Avaliadas	49
4.5	Resultados dos Testes e Desenvolvimento Iterativo	49
4.5.1	Primeira Leva: Protótipo Inicial (Abril de 2026)	49
4.5.2	Segunda Leva: Protótipo Intermédio (Maio de 2026)	50
4.5.3	Terceira Leva: Protótipo Final (Junho de 2026)	50
4.5.4	Feedback Qualitativo e Usabilidade Visual	52
4.6	Discussão e Limitações do Estudo	53
4.6.1	Discussão Crítica do Desenvolvimento	53
4.6.2	Limitações Metodológicas da Avaliação	53
4.6.3	Limitações Tecnológicas e Funcionalidade de IA	53
5	Conclusão e Trabalho Futuro	55
5.1	Conclusão	55
5.1.1	Porquê Escolher o Plot in a Pot?	56
5.2	Trabalho Futuro	56
	<i>Bibliography</i>	59
	Apêndices	
A	Manual de Utilização e Instalação	64
A.1	Requisitos do Sistema e Instalação Local	64
A.1.1	Instalação do Editor Web	64
A.1.2	Configuração do Servidor de IA Local (Ollama)	65
A.2	Manual de Utilização do Editor Gráfico	65
A.2.1	O Canvas Interativo	65
A.2.2	Criação e Edição de Nós Narrativos (Cenas)	66
A.2.3	Criação de Ligações e Arestas de Escolha	66
A.2.4	Agrupamento em Zonas (Capítulos)	66
A.3	Programação de Lógica Narrativa e Variáveis	67
A.3.1	Criação de Variáveis (Figma-Style Tokens)	67
A.3.2	Edição de Lógica condicional por Código	67
A.3.3	Editor de Lógica Estruturada	68
A.4	Tradução e Localização Multi-idioma	68
A.4.1	Configurar Idiomas	68
A.4.2	Editar Textos na Matriz	68
A.4.3	Importação e Exportação de Ficheiros Comma-Separated Values (CSV)	68
A.5	Validação Lógica e Simulação de Jogo	69
A.5.1	O Painel de Erros de Validação Estática	69

A.5.2	Simulação Ativa (PlayMode) e Testes com o Smart Walker	69
A.6	Operações de Ficheiros e Publicação do Projeto	69
A.6.1	Importar Ficheiros Twee 3	69
A.6.2	Exportar e Compilar a História	70
B	Guia de Sintaxe e Modelagem de Macros Lógicas	71
B.1	Estrutura de um Documento Twee 3	71
B.1.1	Passagens Especiais de Sistema	71
B.1.2	Passagens Narrativas (Cenas)	72
B.2	Sintaxe de Macros Lógicas (SugarCube)	72
B.2.1	Macro de Atribuição: <<set>>	72
B.2.2	Macro de Controlo Condicional: <<if>>	72
B.3	Padrões Comuns de Modelação Narrativa	73
B.3.1	Padrão 1: Inventário e Portas Trancadas	73
B.3.2	Padrão 2: Contador de Tentativas e Fim de Jogo	73
B.3.3	Padrão 3: Caminhos Ocultos e Nós Secretos	74
C	Guião de Testes e Questionário de Avaliação	75
C.1	Guião de Tarefas de Teste	75
C.2	Questionário System Usability Scale (SUS)	76

Lista de Figuras

2.1	Diferentes tipologias de grafos e as suas representações em matrizes de adjacência.	8
2.2	Interface de utilizador do Twine com exibição gráfica de passagens e conexões.	10
2.3	Editor Inky para a linguagem Ink, com o painel de código (esquerda) e teste (direita).	11
2.4	Interface visual do Yarn Spinner integrada num ambiente de desenvolvimento.	12
2.5	Interface do editor visual do Narrative Flow com ligação de blocos lógicos. . .	12
2.6	Interface do Articy Draft X para planeamento profissional de histórias interativas.	13
3.1	Diagrama de blocos detalhado da arquitetura conceptual e fluxo de dados. . .	19
3.2	Exemplo de uma Zona de Agrupamento Visual com nós de cena filhos no canvas.	24
3.3	Exemplo de aresta suavizada com caminho curvado personalizável no canvas.	25
3.4	Interface gráfica global do editor (<i>Plot in a Pot</i>), ilustrando a identidade visual neo-brutalista de alto contraste com contornos espessos e sinalização redundante para acessibilidade.	26
4.1	Interface gráfica do canvas de grafos do Plot in a Pot, demonstrando a estética neo-brutalista de alto contraste com nós de cena e agrupamentos visuais (Zonas).	35
4.2	Diálogo de gestão de variáveis globais (VariablesModal), apresentando os tokens de dados e a renomeação segura baseada em Expressões Regulares (Regex).	40
4.3	Matriz de localização integrada no editor, exibindo a grelha de chaves de texto e a respetiva tradução direta para os idiomas selecionados.	40
4.4	Ecrã da sandbox de simulação (PlayMode), ilustrando a execução em tempo real da história com o painel de depuração de variáveis lógicas.	42
4.5	Interface do editor visual de lógica estruturada (VisualBlocksEditor), dividindo as passagens em painéis de prosa, setters de variáveis e escolhas condicionais.	43

Lista de Tabelas

2.1	Comparação de características principais entre motores de narrativa interativa.	13
3.1	Mapeamento de operadores lógicos SugarCube para JavaScript.	31
4.1	Complexidade algorítmica teórica dos subsistemas nucleares.	47
4.2	Resultados empíricos de benchmarks de desempenho (em milissegundos). . .	48
4.3	Taxa de sucesso e tempo de conclusão (em minutos) das tarefas por utilizador (Leva Final).	51
4.4	Pontuação obtida no questionário System Usability Scale (SUS) na leva final.	52

Siglas

API	Application Programming Interface
AST	Árvore de Sintaxe Abstrata (<i>Abstract Syntax Tree</i>)
BFS	Pesquisa em Largura (<i>Breadth-First Search</i>)
CSV	Comma-Separated Values
DAG	Grafo Direcionado Acíclico (<i>Directed Acyclic Graph</i>)
DFS	Pesquisa em Profundidade (<i>Depth-First Search</i>)
ESTG	Escola Superior de Tecnologia e Gestão
NI	Narrativa Interativa
IFTF	Interactive Fiction Technology Foundation
JSON	JavaScript Object Notation
PiaP	Plot in a Pot
Regex	Expressões Regulares
SPA	Single Page Application

1

Introdução

O desenvolvimento de jogos e narrativas interativas tem vindo a ganhar uma importância crescente no panorama do entretenimento e da educação. Diferente do cinema ou da literatura tradicional, uma Narrativa Interativa (NI) coloca o utilizador no centro da tomada de decisões, permitindo-lhe moldar o rumo dos acontecimentos através das suas escolhas. Esta não-linearidade narrativa, no entanto, introduz uma complexidade acrescida na escrita e no planeamento das histórias, exigindo ferramentas capazes de gerir fluxos de informação complexos e lógica de estados dinâmicos.

Neste contexto surge o *Plot in a Pot*, um editor visual e simulador de narrativas interativas. O projeto foi concebido para resolver os principais desafios enfrentados pelos autores de histórias não-lineares, combinando uma interface visual de alto desempenho baseada em grafos direcionados com um motor de simulação que suporta a execução de código e lógica em tempo real.

1.1 Objeto do Trabalho

O objeto deste projeto consiste no desenvolvimento do Plot in a Pot (PiaP), uma aplicação e ambiente de desenvolvimento concebido para a criação, estruturação, simulação e tradução de narrativas interativas e não-lineares. O sistema foca-se na integração e suporte nativo ao ecossistema do motor Twine, operando especificamente com o formato de ficheiros Tweep (versão 3) e com a macro-linguagem SugarCube. Através de uma interface gráfica baseada em grafos, a ferramenta permite a autores e *narrative designers* gerir ramificações complexas e bases de dados de localização de uma forma unificada.

1.2 Justificação e Pertinência do Tema

O *design* de narrativas para jogos não-lineares enfrenta frequentemente problemas severos de escala, onde árvores de diálogo e escolhas crescem exponencialmente, tornando-se difíceis de procurar e propensas a erros lógicos. Embora o Twine seja uma das ferramentas de referência da comunidade independente e académica (Interactive Fiction

Technology Foundation, 2026) para a prototipagem rápida de NI, o desenvolvimento de projetos profissionais de grande escala é severamente limitado pela falta de ferramentas nativas para a gestão multi-idioma (localização) e pela ausência de validadores estáticos de fluxo que previnam caminhos mortos antes da compilação.

O PiaP justifica-se ao resolver esta lacuna, introduzindo uma estética neo-brutalista de alto contraste que otimiza o espaço de trabalho e melhora a acessibilidade de autores com limitações de visão (Babich, 2023; Loranger, 2022), permitindo isolar cadeias de texto rígidas (*hardcoded*) através de um sistema abstrato de chaves de tradução dinâmicas e oferecendo segurança matemática no mapeamento de rotas narrativas.

1.3 Objetivos do Trabalho

1.3.1 Objetivo Geral

Desenvolver uma plataforma *web* modular que unifique a edição visual de grafos de histórias interativas Twee com um motor integrado de localização multi-idioma e validação de caminhos lógicos.

1.3.2 Objetivos Específicos

Para a concretização do objetivo geral, definiram-se os seguintes objetivos específicos:

1. **Interface Gráfica Baseada em Grafos:** Desenvolver um *canvas* interativo de alto desempenho que utilize uma estética de alto contraste baseada no estilo *neo-brutalista*. Este ambiente deve permitir a manipulação visual de nós narrativos, blocos de agrupamento por zonas, *scripts* em JavaScript e estilização em CSS.
2. **Encaminhamento Geométrico de Arestas:** Desenvolver um algoritmo de vetorização que utilize *waypoints* interativos nas ligações entre nós, prevenindo a sobreposição visual de caminhos e otimizando a organização do espaço de trabalho.
3. **Sincronização Bidirecional com Formato Twee3:** Construir um interpretador (*parser*) capaz de traduzir a sintaxe de Twee3 (`[[Link|Destino]]`) na topologia visual do grafo, gerindo a conversão de coordenadas espaciais de forma transparente.
4. **Módulo de Internacionalização Nativo:** Desenvolver uma matriz de localização flexível que permita a tradução direta por células e suporte a importação e exportação de dados em formato CSV, centralizando a gestão multi-idioma.
5. **Análise Estática de Integridade Narrativa:** Implementar um motor de validação baseado em listas de adjacência para auditar o grafo, detetando de forma automática anomalias estruturais como nós isolados, becos sem saída lógicos ou ciclos fechados de *loops* infinitos.
6. **Automação Narrativa por Inteligência Artificial:** Projetar uma arquitetura de inferência local que utilize Modelos de Linguagem de Grande Escala (*LLMs*)

para processar narrativas escritas em linguagem natural e convertê-las automaticamente na estrutura formal de grafos do motor de jogo.

7. **Ambiente de Execução e Aprendizagem:** Criar um simulador integrado (*PlayMode*) para testes imediatos do fluxo narrativo e um módulo pedagógico interativo (*Playable Tutorial*) para a introdução assistida às regras e funcionalidades do editor.

1.4 Estrutura do Documento

Este relatório está estruturado em cinco capítulos principais. Além desta introdução, o documento apresenta:

- **Capítulo 2: Fundamentação Teórica e Estado da Arte**, onde se efetua uma análise comparativa das ferramentas existentes no mercado e se descrevem os conceitos matemáticos de teoria de grafos aplicados à narrativa.
- **Capítulo 3: Conceção e Desenvolvimento**, detalhando a arquitetura do software, a especificação Tweep3, o modelo de lista de adjacências bidirecional, os modos de visualização e acessibilidade, e a simulação lógica.
- **Capítulo 4: Implementação e Avaliação**, que descreve o cronograma de desenvolvimento, a estrutura física do código e detalhes de implementação da aplicação, bem como a metodologia e resultados dos testes de utilizador realizados para validar o sistema.
- **Capítulo 5: Conclusão e Trabalho Futuro**, onde são sumariados os resultados alcançados, as limitações da implementação e as propostas para futuros desenvolvimentos.

2

Fundamentação Teórica e Estado da Arte

Neste capítulo são expostos os fundamentos teóricos relativos à modelação matemática de fluxos narrativos interativos, bem como a análise do estado da arte sobre as ferramentas de autoria existentes, destacando os seus pontos fortes e limitações.

2.1 Contexto Teórico

A criação de narrativas não lineares enfrenta frequentemente a “Explosão de Complexidade”, na qual os modelos tradicionais baseados em sequências ou cadeias falham em produzir narrativas logicamente consistentes, resultando muitas vezes em conteúdo repetitivo ou contradições causais (Chen et al., 2021). A poluição visual nas ferramentas tradicionais surge porque fluxogramas simples ou cadeias de eventos não conseguem capturar de forma eficaz as ligações densas e multidimensionais entre batidas narrativas, conduzindo a problemas de “alucinação” em sistemas automatizados, nos quais os eventos previstos carecem de precisão ou de completude (Li et al., 2018). Sem uma estrutura formal, os designers enfrentam dificuldades significativas na gestão de percursos ramificados, o que pode conduzir a evoluções narrativas incoerentes que são cognitivamente exigentes de depurar ou verificar manualmente (Mormoris, 2024).

Para ultrapassar estas limitações estruturais, *frameworks* recentes recorrem a *Narrative Event Evolutionary Graphs* (NEEG) e a *Event-centric Temporal Knowledge Graphs*, de forma a fornecer uma representação estruturada da evolução dos eventos (Li et al., 2018; Yan et al., 2023). Ao tratar as histórias como grafos direcionados com relações temporais e causais explícitas, estes sistemas conseguem garantir um fluxo lógico e prevenir as inconsistências encontradas em designs manuais tipo “esparguete” (Chen et al., 2021; Li et al., 2018; Tang et al., 2022). A implementação de uma *framework* deste tipo permite a utilização de conjuntos de exclusão mútua e métricas de coerência para assegurar que os eventos simultâneos são lógicos e que cada batida narrativa permanece alcançável dentro da hierarquia pretendida (Chen et al., 2021). Esta mudança para uma

organização baseada em grafos transforma as ferramentas narrativas de meras aplicações de desenho em validadores lógicos ativos, garantindo consistência global através das propriedades matemáticas formalizadas da interação entre eventos (Chen et al., 2021; Tang et al., 2022).

Tendo em conta o facto de ainda não existirem normas estabelecidas, ao formalizar uma narrativa como um Conjunto Parcialmente Ordenado (Poset)¹, este projeto pode ir além da simples visualização para criar um Validador Lógico que assegura a integridade estrutural ao nível de arquitetura.

2.2 Conceitos Fundamentais de Narrativas Não-Lineares

Antes de abordar a modelação computacional, importa clarificar a evolução e os conceitos essenciais que regem as narrativas não-lineares. Ao contrário do modelo tradicional, cuja progressão textual é puramente sequencial do início ao fim, uma **narrativa não-linear** caracteriza-se pela fragmentação do discurso e pela existência de múltiplos percursos de leitura.

Do ponto de vista estrutural, e independentemente da tecnologia utilizada, qualquer história ramificada assenta em três elementos fundamentais:

- **Cenas (Nós):** Os fragmentos autónomos de conteúdo. sejam estes textuais, visuais ou sonoros. Contêm a substância dramática exposta ao leitor num determinado momento da história.
- **Escolhas (Ligações):** Os mecanismos de bifurcação expostos ao longo da narrativa que convidam o utilizador a realizar uma escolha, determinando o vetor de transição para o fragmento de conteúdo seguinte.
- **Memória Narrativa (Estado):** O conjunto de dados acumulados que regista o histórico de ações, a posse de objetos ou os parâmetros quantificáveis do percurso do leitor. Esta memória altera o contexto da obra em tempo real, ditando se certos caminhos se tornam acessíveis ou bloqueados no futuro.

Para ilustrar estas dinâmicas de forma clara, considere-se o seguinte exemplo narrativo:

A sua personagem acorda numa clareira envolta por uma floresta densa e percebe que acabou de amanhecer. Ao olhar em redor com mais detalhe, nota dois elementos que lhe chamam a atenção: uma casa abandonada por entre as árvores e um trilho no lado oposto. A casa parece trancada, mas pressente que conseguirá abrir a porta se encontrar algo para forçar a entrada. O trilho, por outro lado, aparenta não oferecer qualquer tipo de bloqueio.

Para onde vai a sua personagem?

- Casa abandonada

¹ As propriedades de reticulado são fundamentais para garantir a integridade lógica da narrativa.

- Trilho oposto

Neste cenário, a descrição inicial da clareira constitui a **Cena** ativa (o nó de partida). As opções dadas ao utilizador representam as **Escolhas** (as ligações ou arestas do grafo) que despoletam a transição estrutural. Se o utilizador optar por seguir o trilho e, mais à frente, recolher uma ferramenta, essa ação é registada na **Memória Narrativa** (o estado). Numa interação futura, caso decida regressar à casa abandonada, o motor avaliará retroativamente essa memória para validar se o caminho para o interior da habitação se deve abrir ou permanecer trancado.

2.3 Teoria de Grafos na Narrativa Interativa

A estrutura básica de uma história interativa assenta na modelação geométrica e lógica de caminhos narrativos. Um fluxo de história pode ser formalmente representado como um Grafo $G = (V, E)$ (Bondy et al., 2008), onde:

- V representa o conjunto de **Vértices** (ou nós), que correspondem a cenas, passagens de texto ou estados lógicos da história.
- E representa o conjunto de **Arestas** (ou ligações direcionadas), que representam as escolhas disponibilizadas ao leitor para transitar de uma cena para a outra.

De seguida, são apresentadas as diferentes estruturas de grafos aplicadas ao design narrativo, a comparação fundamental entre fluxos acíclicos e com ciclos, e a fundamentação matemática que justifica a necessidade de ferramentas automatizadas para mitigar a explosão de complexidade lógica.

2.3.1 Tipos de Grafos e Aplicação Narrativa

Dependendo da natureza do jogo ou da história, utilizam-se diferentes tipos de estruturas de grafos (ilustrados na Figura 2.1):

1. **Grafo Direcionado (Directed Graph):** Uma estrutura onde as ligações têm direção, ou seja, a existência de um caminho de $A \rightarrow B$ não implica o retorno de $B \rightarrow A$. Numa matriz de adjacência (uma tabela bidimensional que mapeia as conexões entre todos os nós do grafo, onde a linha i e a coluna j indicam a existência de uma ligação do nó i ao nó j), isto traduz-se numa estrutura não-simétrica:

$$M[i][j] \neq M[j][i]$$

Representa o fluxo narrativo puro, onde cada escolha conduz o leitor numa direção específica.

2. **Grafo Não-Direcionado (Undirected Graph):** Representa relações simétricas e bidirecionais ($A - B$). Embora seja menos comum em narrativa linear, é útil para modelar redes de relacionamento social entre personagens ou conexões espaciais

de livre exploração (p. ex., salas adjacentes de uma masmorra). Na matriz de adjacência, a estrutura é simétrica:

$$M[i][j] = M[j][i]$$

3. **Grafo de Conhecimento (Knowledge Graph):** Modela relações semânticas complexas entre entidades (p. ex., “personagem conhece objeto”, “porta depende de chave”). Exige múltiplas matrizes de adjacência ou uma estrutura tridimensional:

$$M_{relation}[i][j]$$

É ideal para jogos com inventários complexos e decisões baseadas em conhecimento indireto acumulado pelo utilizador.

4. **Grafo Ponderado (Weighted Graph):** Atribui valores (pesos) às arestas:

$$M[i][j] = \text{peso}$$

Estes pesos podem representar custos de recursos (tempo, pontos de vida), custos emocionais do personagem, ou a probabilidade de uma escolha ter sucesso.

2.3.2 Modelação por Grafo Controlado por Estados

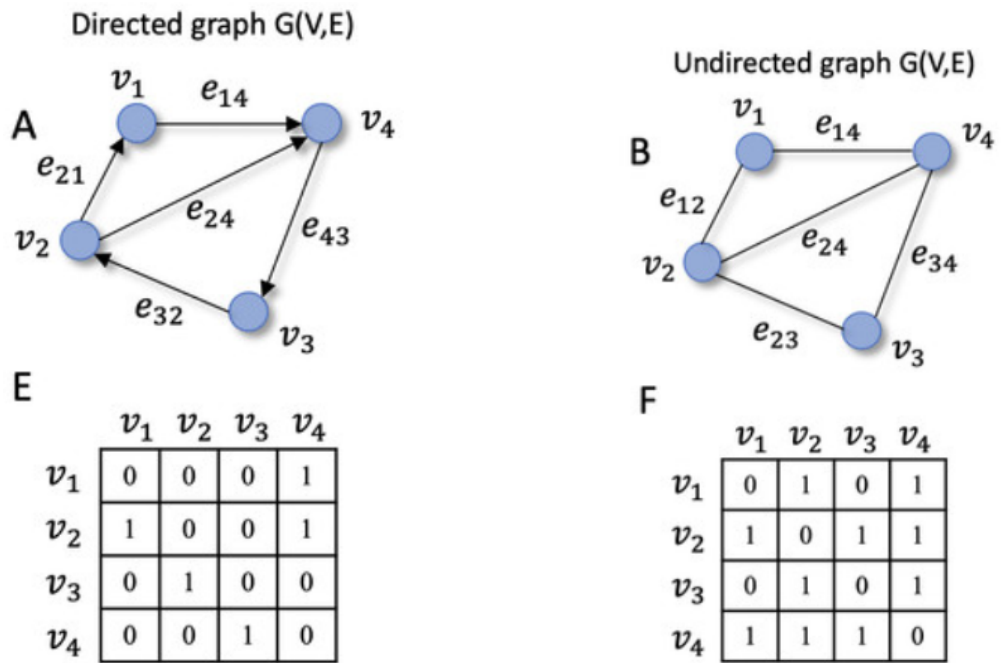
No desenvolvimento de sistemas narrativos, é frequente restringir a estrutura a Grafos Direcionados Acíclicos (*DAGs*) para garantir um fluxo estritamente unidirecional. Contudo, essa abordagem impede a criação de mecânicas essenciais como *hubs* centrais de exploração ou ciclos de passagem de tempo baseados em ações recorrentes.

Por este motivo, o *Plot in a Pot* admite a existência de ciclos, mas afasta-se do modelo convencional de grafo direcionado simples onde uma aresta une rigidamente um nó *A* a um nó *B*. Em vez disso, a aplicação implementa um modelo de **Grafo Controlado por Estados**. No contexto da ferramenta, as ligações entre as cenas não são estáticas; o caminho efetivamente percorrido depende diretamente das macros condicionais (p. ex. `<<if>>`) e do estado atual das variáveis do utilizador.

Isto significa que a estrutura real da história não é determinada apenas pelo visual do *canvas*, mas sim pela computação em tempo de execução. Uma única escolha desenhada no editor pode direcionar o jogador para múltiplos destinos alternativos, dependendo estritamente do valor das variáveis em memória. Esta flexibilidade exige algoritmos de validação em segundo plano (como pesquisas BFS e Pesquisa em Profundidade (Depth-First Search) (DFS) (Cormen et al., 2009)) adaptados para analisar a conectividade considerando o impacto destas variáveis antes da compilação final.

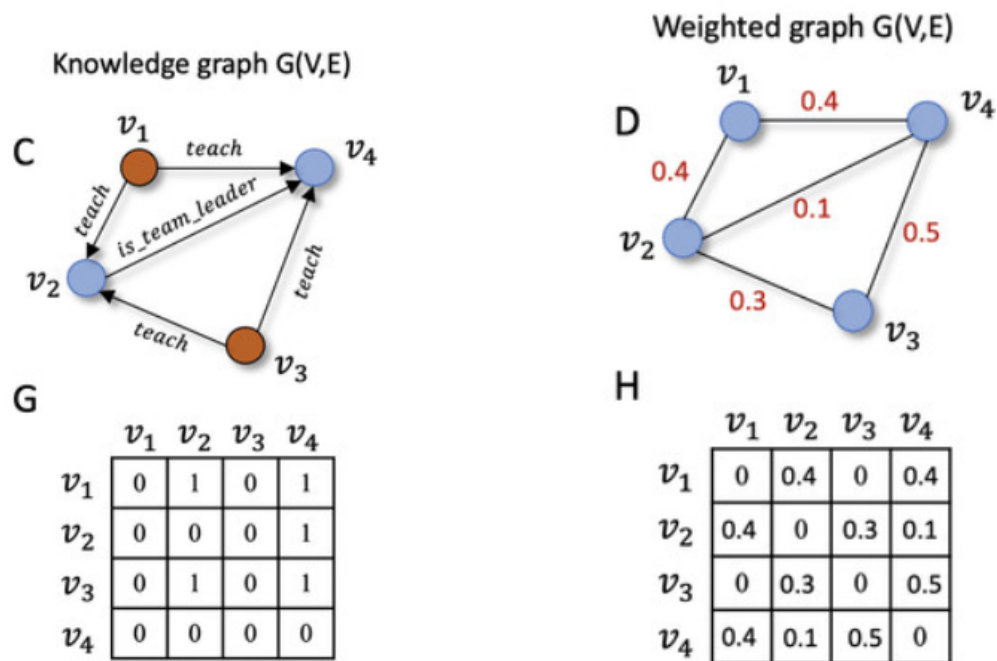
2.3.3 O Desafio da Explosão Combinatória

A necessidade de validação automatizada no desenvolvimento de narrativas interativas prende-se com o fenómeno da **explosão combinatória**. À medida que o autor adiciona



(a) Grafo Direcionado (A) e respetiva Matriz de Adjacência (E).

(b) Grafo Não-Direcionado (B) e respetiva Matriz de Adjacência (F).



(c) Grafo de Conhecimento (C) e respetiva Matriz de Adjacência (G).

(d) Grafo Ponderado (D) e respetiva Matriz de Adjacência (H).

Figura 2.1: Diferentes tipologias de grafos e as suas representações em matrizes de adjacência.

novas escolhas e bifurcações à história, o número de caminhos lógicos possíveis cresce de forma exponencial.

Este problema agrava-se com a introdução de ciclos (que permitem regressar a cenas anteriores) e variáveis de estado (como inventários ou escolhas morais). Em projetos de grande dimensão, a complexidade resultante torna impossível a um autor humano auditar manualmente todas as combinações de escolhas e validar se existem caminhos bloqueados, becos sem saída ou loops infinitos onde o utilizador possa ficar preso. Justifica-se, assim, o desenvolvimento do motor de verificação estática do *Plot in a Pot*, capaz de explorar automaticamente todas as possibilidades lógicas do projeto e alertar o criador sobre anomalias na escrita em tempo real.

2.4 Software de Criação de Histórias não Lineares

O conceito de NI refere-se a narrativas estruturadas onde a progressão do texto não é linear, dependendo diretamente das escolhas e ações submetidas pelo utilizador para navegar caminhos alternativos (Montfort, 2005). Embora a tecnologia digital tenha popularizado o termo, a literatura não-linear física remonta a precursores como os livros-jogo (*gamebooks*) e a série *Choose Your Own Adventure* popularizada nas décadas de 1970 e 1980 (Packer, 1984), bem como a experiências literárias ramificadas pioneiras (p. ex., o conto de Jorge Luis Borges em 1941) (Borges, 1941; Aarseth, 1997). Com a expansão recente do mercado de videojogos independentes (Juul, 2005; Adams, 2014) e o desenvolvimento da narratologia computacional (Tang et al., 2022; Yan et al., 2023), emergiram várias ferramentas de autoria gráfica e linguagens de scripting para suportar a criação de estruturas ramificadas (Interactive Fiction Technology Foundation, 2026).

Apresenta-se, de seguida, uma análise detalhada das principais referências do mercado académico e profissional.

2.4.1 Twine

Lançado em 2009 por Chris Klimas (Klimas, 2009), o Twine é a ferramenta mais popular na comunidade de NI baseada em texto.

- **Vantagens:** Extremamente acessível (sem barreira de programação para escrita básica); visualização clara baseada em nós gráficos no canvas; open-source e gratuito (distribuído sob a licença GPL v3, permitindo qualquer utilização, incluindo comercial, sem restrições); rápido para prototipagem.
- **Limitações:** A interface visual torna-se desorganizada e lenta em projetos muito grandes (efeito “prato de esparguete”); a lógica de estado nativa é limitada; a integração com motores de jogos externos (como Unity ou Unreal) é complexa.

O fluxo de trabalho central do Twine assenta na criação de passagens (segmentos individuais da história) e na sua ligação através de hiperligações que representam escolhas do jogador. Estas passagens são exibidas como caixas num canvas com setas a indicar os

caminhos narrativos (Klimas, 2009), conforme ilustrado na Figura 2.2. Este paradigma mapeia-se diretamente para a representação em canvas de grafos interativos que constitui o núcleo visual do PiaP, tornando a correspondência entre o modelo de dados interno e a interface imediata e semanticamente coerente.

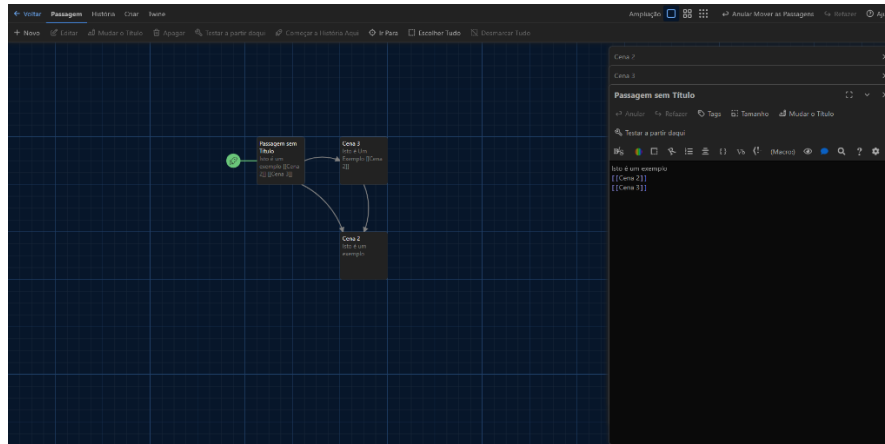


Figura 2.2: Interface de utilizador do Twine com exibição gráfica de passagens e conexões.

2.4.2 Ink (Inkle Studios)

O Ink (Inkle Studios, 2016) é uma linguagem textual de scripting criada pela Inkle Studios (criadores de jogos como *80 Days*). A sua escrita é efetuada no editor Inky (apresentado na Figura 2.3), que oferece pré-visualização em tempo real da narrativa criada.

- **Vantagens:** Altamente poderosa para o controlo de estados narrativos através de variáveis locais e globais, condicionais que ativam escolhas dinamicamente e gestão de ramificações complexas assente em sub-nós lógicos (*stitches* e *gathers*); excelente integração técnica com motores de jogo (tais como Unity e Unreal Engine, que contam com suporte oficial da editora); licenciada sob a licença MIT (totalmente livre); escala muito bem para projetos de grande envergadura.
- **Limitações:** Não é uma ferramenta visual, operando estritamente através de ficheiros de texto estruturados. A ausência de um canvas ou representação espacial de grafos de nós dificulta o planeamento geográfico da narrativa, tornando impossível visualizar de forma imediata o fluxo geral ou detetar de forma intuitiva incoerências, ciclos infinitos e becos sem saída sem uma análise exaustiva e manual de milhares de linhas de código por parte de autores não-técnicos; exige fortes conhecimentos de lógica de programação.

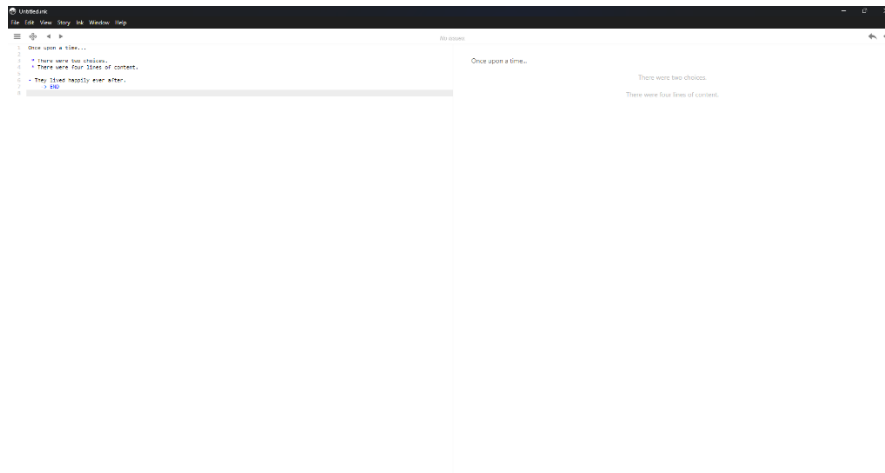


Figura 2.3: Editor Inky para a linguagem Ink, com o painel de código (esquerda) e teste (direita).

2.4.3 ChoiceScript

Desenvolvido pela Choice of Games, o ChoiceScript é uma linguagem de programação simples e baseada em texto para a escrita de romances interativos baseados em escolhas.

- **Vantagens:** Sintaxe simples e linear de fácil compreensão para escritores; possui excelentes ferramentas de teste automático (como `quicktest` e `randomtest`) que facilitam a validação de caminhos e a robustez lógica de romances longos.
- **Limitações:** Não é uma ferramenta visual; o formato de ficheiro é proprietário; a sua licença (CSL) proíbe explicitamente a utilização comercial sem autorização prévia e acordo de distribuição comercial com a Choice of Games (Choice of Games, 2026), limitando a autonomia de autores independentes.

2.4.4 Yarn Spinner

O Yarn Spinner (Secret Lab, 2015) é uma ferramenta focada na integração de diálogos em jogos interativos com motor gráfico (ilustrada na Figura 2.4).

- **Vantagens:** Sintaxe simples (semelhante a scripts de teatro); suporte nativo para variáveis; excelente portabilidade direta para Unity e Unreal Engine.
- **Limitações:** Carece de um planeador visual abstrato robusto fora do motor de jogo principal; não é adequado para desenhar fluxogramas abstratos de histórias antes da fase de produção do jogo.

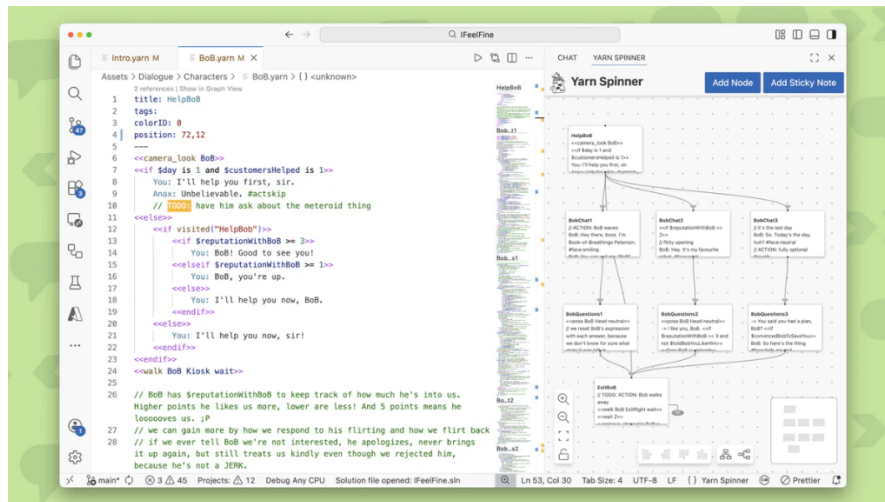


Figura 2.4: Interface visual do Yarn Spinner integrada num ambiente de desenvolvimento.

2.4.5 Narrative Flow

O Narrative Flow (NarrativeFlow, 2023) é uma ferramenta emergente que tenta colmatar a lacuna entre Twine e Ink através de um editor visual baseado em cartões (Figura 2.5).

- **Vantagens:** Oferece melhor organização estrutural do que o Twine e uma visualização mais clara que o Ink; ajuda a gerir a consistência da escrita.
- **Limitações:** É um software pago, com menor maturidade técnica e uma comunidade de utilizadores consideravelmente mais reduzida.

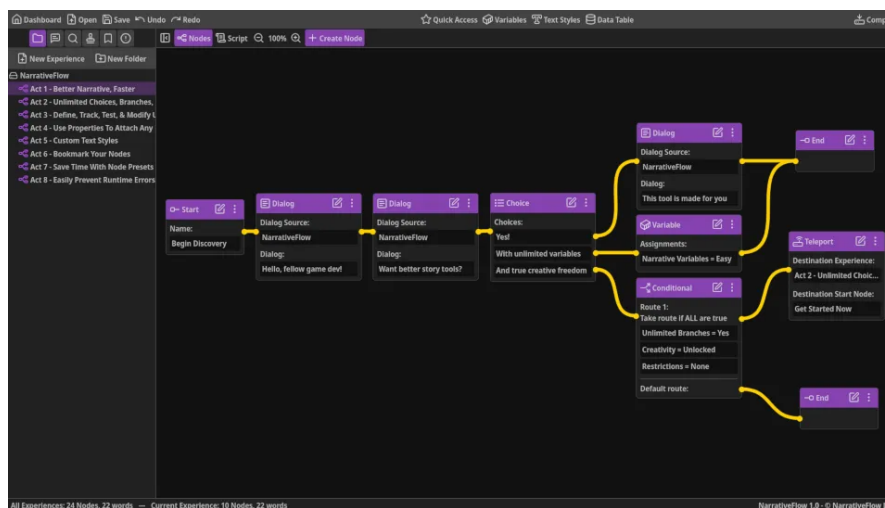


Figura 2.5: Interface do editor visual do Narrative Flow com ligação de blocos lógicos.

2.4.6 Articy Draft

O Articy Draft (Articy Software, 2010) é a solução comercial líder da indústria de videojogos AAA para planeamento de narrativas e bases de dados de escrita (apresentado na Figura 2.6).

- **Vantagens:** Altamente estruturado e profissional; gestão integrada de personagens, inventários e diálogos; escala com eficácia para equipas de desenvolvimento de grandes dimensões.
- **Limitações:** Curva de aprendizagem acentuada; software dispendioso (licença comercial paga); excessivamente complexo e pesado para autores independentes ou projetos pequenos.

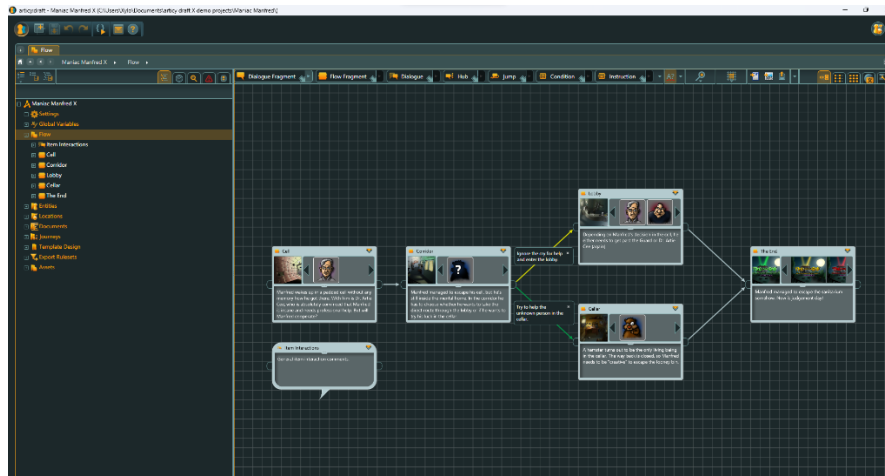


Figura 2.6: Interface do Articy Draft X para planeamento profissional de histórias interativas.

2.4.7 Análise Comparativa

A fim de estruturar a comparação entre as diferentes abordagens, apresenta-se na Tabela 2.1 uma síntese comparativa das ferramentas baseada nos seus recursos, suporte a lógica de jogo e preços.

Tabela 2.1: Comparação de características principais entre motores de narrativa interativa.

Característica	Twine	Ink	Choice-Script	Yarn Spinner	Narrative Flow	Articy Draft
Edição Visual	Sim	Não	Não	Sim	Sim	Sim
Variáveis e Estado	Sim	Sim	Sim	Sim	Sim	Sim
Lógica Condicional	Sim	Sim	Sim	Sim	Sim	Sim
Verificação Estática	Não	Não	Não	Não	Não	Não
Integração (Engines)	Não	Sim	Não	Sim	Sim	Sim
Gratuito e Aberto	Sim	Sim	Sim	Sim	Não	Não

A seleção dos critérios de avaliação apresentados na Tabela 2.1 justifica-se pela necessidade de analisar de que forma as soluções existentes respondem aos desafios práticos da autoria e produção de NI:

- **Edição Visual:** Identifica se o motor oferece uma representação gráfica em forma de canvas ou nós para visualização e mapeamento direto dos caminhos narrativos (essencial para autores não-técnicos).

- **Variáveis e Lógica Condicional:** Avalia o suporte para manter uma memória narrativa (estado) e a capacidade de restringir escolhas dinamicamente com base nas ações do leitor.
- **Verificação Estática:** Determina se a ferramenta realiza auditorias automáticas para detetar becos sem saída, ciclos sem fim ou incoerências lógicas em tempo de desenho, uma funcionalidade tradicionalmente ausente nas soluções existentes.
- **Integração com Engines:** Mede a portabilidade da narrativa para motores gráficos como Unity, Unreal Engine ou Godot, indispensável para produções de jogos modernos.
- **Gratuito e Aberto:** Reflete a acessibilidade financeira e a ausência de restrições comerciais ou licenciamento fechado na distribuição das obras criadas.

2.5 O Formato Twee3 e Linguagens de Scripting Narrativo

Para além da representação visual em grafos, o ecossistema de narrativa interativa apoia-se em formatos de serialização de texto que permitem aos autores escrever e armazenar as suas obras em ficheiros de texto simples (.txt ou .twee). O padrão da indústria para esta representação é o **Twee3**, uma especificação aberta mantida pela *Interactive Fiction Technology Foundation* (IFTF) (Interactive Fiction Technology Foundation, 2026).

O formato Twee3 organiza o ficheiro narrativo através de blocos lógicos denominados **Passagens** (Passages). Cada passagem começa com um identificador de início (duas frações de dois pontos seguidas do título da passagem):

Listing 2.1: *Estrutura básica de uma passagem em Twee3.*

```

1 :: Floresta
2 Acordas numa floresta densa e silenciosa.
3 Ves um trilho a tua frente.
4
5 [[Seguir pelo trilho->Trilho]]

```

2.5.1 Sintaxe de Ligações e Transições

No Twee3, as bifurcações narrativas e as escolhas do leitor são representadas por parênteses retos duplos. Existem duas sintaxes principais para definir transições:

- **Ligação Direta:** [[Nome da Passagem]], onde o texto da escolha apresentado ao leitor é idêntico ao título da passagem de destino.
- **Ligação Rotulada:** [[Texto da Escolha->Destino]], onde o texto visível é separado do identificador da passagem por uma seta.

2.5.2 Metadados Geométricos e Etiquetas (Tags)

Para além do conteúdo narrativo e das ligações, a especificação normativa do Twee3 suporta a inclusão de metadados em formato JavaScript Object Notation (JSON) e de etiquetas (*tags*) diretamente no cabeçalho das passagens. O cabeçalho estendido obedece à seguinte estrutura:

Listing 2.2: *Exemplo de cabeçalho estendido com etiquetas e metadados geométricos em Twee3.*

```
1 :: Nome da Passagem [etiqueta1 etiqueta2]
   {"position":"100,200","size":"100,100"}
```

Onde:

- **Etiquetas (Tags):** Declaradas entre parênteses retos ([...]) após o título da passagem, servem para catalogar ou aplicar propriedades especiais de visualização ou lógica.
- **Metadados JSON:** Delimitados por chavetas ({...}), contêm dados estruturados de persistência da história. As propriedades padrão para a representação visual em editores gráficos são a `position` (coordenadas X,Y absolutas no ecrã) e a `size` (largura e altura físicas da caixa no canvas).

2.5.3 Destinos Dinâmicos vs. Validação Estática

A especificação normativa do Twee3 permite que os autores utilizem expressões lógicas ou variáveis dinâmicas como destinos de transição (por exemplo, `[[Seguir em frente|$destino]]` ou `[[Voltar|Trilho + $passagemAnterior]]`). Em tempo de execução, o motor de jogo resolve o valor da variável e executa o redirecionamento para o nó correspondente.

Embora esta flexibilidade dinâmica traga vantagens de modularidade, ela impede a representação estática do fluxo narrativo sob a forma de um grafo determinista e inviabiliza análises automáticas de integridade antes da execução. Visto que o destino de um nó não pode ser determinado sem correr o jogo passo a passo, o compilador fica impossibilitado de construir a lista de adjacências e mapear as arestas visuais.

Por esta razão, o PiaP introduz uma restrição sintática ao formato Twee3: todos os destinos de ligação devem ser estáticos e literais (isto é, referenciar textualmente o nome de uma passagem existente no grafo). Ao abdicar desta funcionalidade dinâmica de transição, a plataforma ganha a capacidade de:

- Desenhar de forma determinista as arestas direcionadas que ligam as passagens no canvas interativo.
- Executar com precisão o algoritmo de validação estática de caminhos baseada em BFS para detetar nós órfãos e becos sem saída lógicos.
- Viabilizar a exploração de estados e simulação inteligente de caminhos efetuada pelo agente *Smart Walker*.

2.5.4 Macros e Lógica Condicional

A especificação Twee3 é independente do formato de exibição (story format) utilizado em tempo de execução (como Harlowe, SugarCube ou Chapbook). Cada um destes formatos fornece um conjunto de **Macros** para manipulação do estado da narrativa:

- **Declaração de Estado:** Permite criar ou modificar variáveis na memória narrativa. Por exemplo, em SugarCube: `<<set $temChave = true>>`.
- **Estruturas Condicionais:** Permitem ocultar ou exibir escolhas e caminhos com base no estado atual das variáveis. Por exemplo:

Listing 2.3: *Exemplo de logica condicional em Twee3.*

```

1 <<if $temChave>>
2     [[Abrir a porta trancada->InteriorCasa]]
3 <<else>>
4     A porta esta trancada e precisas de algo para a forçar.
5 <</if>>

```

Esta estrutura híbrida — que combina geometria de grafos, marcação de texto simples e scripting condicional — é a especificação que o *Plot in a Pot* adota nativamente, permitindo importar ficheiros externos de outras ferramentas (como o Twine) e exportar o trabalho final num formato de texto aberto e interoperável.

2.6 Integração de Modelos de Linguagem de Grande Escala (LLMs) Locais

Com o avanço recente no campo da Inteligência Artificial Generativa, a integração de Modelos de Linguagem de Grande Escala (*Large Language Models* - LLMs) tornou-se um recurso valioso para auxiliar autores na expansão automática e co-criação de narrativas interativas. No entanto, para garantir a viabilidade prática do projeto PiaP sem dependências financeiras ou de conectividade externa, focou-se a pesquisa em modelos de execução local e gratuita.

2.6.1 Análise de Famílias de Modelos e Recomendações

Foram analisadas três das principais famílias de modelos em open-weight ajustados para instruções, avaliando a sua capacidade e adequabilidade para autoria de NI:

1. Família Llama 3 e 3.1 (Meta AI):

- **Versão Recomendada:** Llama 3.1 (8B).
- **Justificação:** Representa um dos modelos leves mais capazes do estado da arte. A versão de 8 mil milhões de parâmetros corre com excelente desempenho na maioria dos computadores modernos equipados com placa gráfica

dedicada ou em arquiteturas Mac Apple Silicon (M1/M2/M3), requerendo investimentos de hardware reduzidos.

- **Vantagem Narrativa:** Demonstra uma capacidade excecional no cumprimento estrito de diretrizes de formatação (*system prompts* rígidos). Esta fiabilidade torna-o ideal para gerar respostas estruturadas (por exemplo, em formato JSON contendo a sugestão de cena e as escolhas de saída), facilitando a integração direta no motor lógico do software.

2. Família Mistral (Mistral AI):

- **Versão Recomendada:** Mistral NeMo (12B) ou Mistral v0.3 (7B).
- **Justificação:** Os modelos desenvolvidos pela empresa europeia Mistral AI são amplamente reconhecidos pela comunidade de escrita criativa pelo seu tom mais orgânico e natural. Adicionalmente, tendem a apresentar menor nível de censura prévia, facilitando a escrita livre em géneros literários como ficção geral, mistério ou terror.
- **Vantagem Narrativa:** Apresentam janelas de contexto extremamente eficientes. A janela de contexto atua como a memória de curto prazo do modelo; numa narrativa ramificada, é fundamental que o LLM recorde o histórico de cenas e as decisões anteriores tomadas pelo utilizador para garantir a coerência semântica global do texto gerado.

3. Gemma 2 (Google):

- **Versão Recomendada:** Gemma 2 (9B).
- **Justificação:** Apesar do seu tamanho compacto de 9 mil milhões de parâmetros, destaca-se pelo seu desempenho elevado em raciocínio abstrato e análises textuais comparativas.
- **Vantagem Narrativa:** É altamente eficaz na realização de resumos e sínteses. Esta capacidade é ideal para funcionalidades auxiliares que leiam um determinado ramo completo da história e gerem um resumo conciso (p. ex., “descreve os principais acontecimentos deste arco narrativo”).

3

Conceção e Desenvolvimento

Este capítulo descreve a arquitetura geral do *Plot in a Pot*, detalhando as decisões de design de dados, o funcionamento dos algoritmos de processamento, as estratégias de visualização e acessibilidade adotadas, e o motor de simulação implementado.

3.1 Arquitetura Geral do Sistema

O sistema foi desenhado segundo uma arquitetura modular dividida em três camadas conceptuais independentes de tecnologia. Esta abordagem garante a separação de responsabilidades entre a recolha de comandos, o processamento de dados e a sua persistência, promovendo a manutenibilidade e a evolução isolada de cada módulo:

1. **Camada de Apresentação (Interface):** Responsável por gerir a interação direta com o utilizador. É constituída por:
 - **Canvas de Grafos:** O espaço visual interativo onde o autor cria, manipula e liga os nós narrativos.
 - **Matriz de Tradução:** Uma grelha estruturada que expõe os dicionários de localização e permite a tradução de chaves de texto em tempo real.
 - **Ecrã do Simulador:** A interface interativa de jogo onde o autor testa e experimenta a narrativa sob o ponto de vista do leitor.
2. **Camada de Processamento Lógico (Núcleo):** Contém a inteligência do sistema e os motores de transformação. É constituída por:
 - **Parser Twee 3:** Módulo encarregue de ler ficheiros de texto estruturados em formato Twee 3 e convertê-los no modelo interno da aplicação, bem como serializar e exportá-los de volta.
 - **Validador Estático:** Motor que executa travessias de grafo (BFS/DFS) para inspecionar inconsistências estruturais, como ciclos sem saída, nós inacessíveis ou hiperligações quebradas.
 - **Motor de Simulação:** Sandbox responsável por inicializar scripts customizados, interpretar condicionais lógicas e controlar o estado de jogo na simula-

ção. Este estado é representado por um conjunto de variáveis lógicas globais e locais que registam as decisões do utilizador (ex: inventário, escolhas de caminhos ou pontuações).

3. **Camada de Dados e Persistência:** Representa o modelo de informação e o armazenamento. É constituída por:

- **Lista de Adjacência:** O modelo de dados em memória que representa a topologia do grafo, as saídas (sucessores) e entradas (predecessores) de cada cena.
- **Dicionários JSON:** A base de dados estruturada que armazena os dicionários de tradução e localização no armazenamento local.

O fluxo de funcionamento da aplicação e a relação de comunicação entre estas camadas encontram-se representados no diagrama detalhado da Figura 3.1.

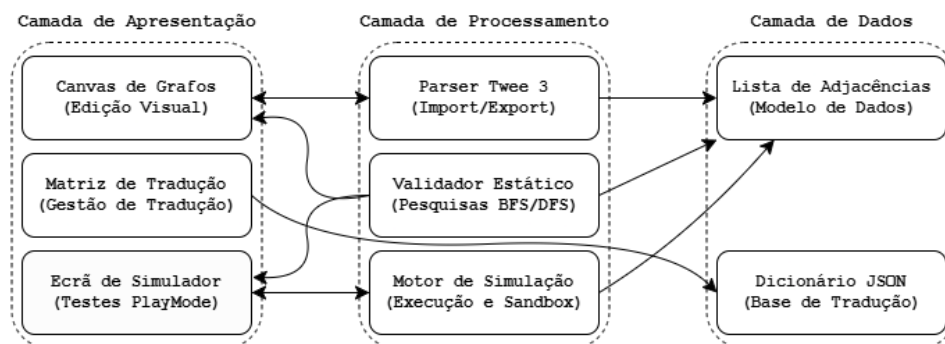


Figura 3.1: Diagrama de blocos detalhado da arquitetura conceptual e fluxo de dados.

3.2 Requisitos do Sistema

Com o intuito de delimitar o âmbito do projeto, o sistema foi estruturado para responder a necessidades fundamentais de modelação visual, internacionalização nativa, análise estática de integridade e inteligência artificial no cliente. Para a materialização prática do PiaP, estas necessidades foram detalhadas sob a forma de requisitos técnicos, divididos entre funcionais (comportamento direto da aplicação) e não funcionais (restrições de qualidade, desempenho e usabilidade).

3.2.1 Requisitos Funcionais

RF01 - Criação e Edição de Nós Narrativos (Cenas): O utilizador deve poder instanciar, arrastar, atualizar (título e corpo de texto das cenas) e remover de forma atómica nós narrativos no canvas bidimensional.

RF02 - Ligações Direcionadas entre Nós (Escolhas): O sistema deve inferir e instanciar arestas direcionadas automaticamente no grafo a partir da análise reativa de hiperligações double-bracket ([[Texto|Destino]]) inseridas pelo autor no texto das passagens.

- RF03 - Parser e Compatibilidade Twee 3:** O sistema deve assegurar a interoperabilidade bidirecional síncrona com a especificação Tweep 3, suportando a importação e exportação de ficheiros com extensão `.tweep`, mantendo a sintaxe e efetuando o mapeamento geométrico das coordenadas espaciais dos nós.
- RF04 - Auditoria e Validação Estática de Fluxo:** A plataforma deve executar análises estáticas e reativas sobre a topologia do grafo em plano secundário, sinalizando inconsistências estruturais como nós órfãos (inacessíveis), becos sem saída (hiperligações para identificadores inexistentes) ou caminhos inacessíveis devido a restrições lógicas.
- RF05 - Matriz de Localização Multi-idioma:** O sistema deve disponibilizar uma matriz de tradução tabular centralizada que associe chaves dinâmicas das passagens a múltiplos dicionários de tradução, permitindo a exportação e importação síncrona de ficheiros em formato CSV em conformidade com a norma RFC 4180.
- RF06 - Simulador em Tempo Real (PlayMode):** A aplicação deve integrar um simulador interativo em tempo real (*PlayMode*) executado sob uma sandbox isolada do DOM, interpretando macros SugarCube, avaliando condições lógicas e executando scripts ou estilos customizados.
- RF07 - Zonas de Agrupamento Visual:** O utilizador deve ser capaz de delimitar contêineres espaciais redimensionáveis (Zonas) para agrupamento lógico e organização de nós de cena, permitindo a sua translação conjunta no canvas.
- RF08 - Importação Inteligente via Conversor LLM:** O sistema deve disponibilizar um assistente de inteligência artificial capaz de converter narrativas lineares em linguagem natural para o formato estruturado Tweep 3, recorrendo à integração assíncrona com modelos de linguagem locais (Ollama) ou APIs externas.

3.2.2 Requisitos Não Funcionais

- RNF01 - Portabilidade e Execução no Cliente (Browser):** A plataforma deve ser executável inteiramente no lado do cliente através de um navegador de internet padrão, sem requerer servidores ou infraestruturas complexas.
- RNF02 - Usabilidade e Acessibilidade Visual:** A interface de grafos deve utilizar a estética neo-brutalista e garantir que a visualização de fluxos (arestas de entrada e saída) seja distinguível sem dependência exclusiva de cor, apoiando utilizadores daltónicos.
- RNF03 - Desempenho e Fluidez:** O canvas do React Flow e a conversão do parser síncrono devem suportar dezenas de nós em simultâneo com latência de interação e atualização abaixo de 100 milissegundos.
- RNF04 - Privacidade e Soberania de Dados:** Todos os dados introduzidos, incluindo ficheiros importados, tabelas de tradução e dados submetidos para inteligência artificial no Ollama local, devem permanecer estritamente no armazenamento local do utilizador para garantir segurança absoluta.

3.3 Métodos, Técnicas e Enquadramento Tecnológico

Para concretizar as especificações da arquitetura conceptual e responder aos requisitos definidos, o projeto apoia-se em metodologias ágeis de prototipagem rápida e numa seleção de tecnologias executadas do lado do cliente. As principais opções metodológicas e técnicas encontram-se estruturadas a seguir:

- **Desenvolvimento da SPA (React 19) e Estilização (Tailwind CSS 3):** A aplicação foi inteiramente estruturada e programada pela equipa como uma Single Page Application (SPA) em **React 19** (Meta Open Source, 2013), tirando partido da nossa experiência prévia com esta tecnologia para gerir reativamente a árvore de componentes da interface. A identidade visual neo-brutalista (de alto contraste e bordas sólidas) foi desenhada e implementada por nós através da aplicação de classes utilitárias de **Tailwind CSS 3** (Tailwind Labs, 2017) nos componentes do editor.
- **Extensões ao Canvas de Grafos (ReactFlow 11):** Em vez de desenvolvermos um motor de desenho vetorial do zero, utilizámos a biblioteca **ReactFlow 11** (webkid.io, 2021) como base para gerir interações de zoom e arrasto. O nosso trabalho consistiu em estender esta infraestrutura básica através do desenvolvimento de nós totalmente customizados de cena (`StoryNode.jsx`) e de agrupamento (`ZoneNode.jsx`), além de arestas interativas personalizadas.
- **Algoritmia de Validação (Teoria de Grafos):** Concebemos e implementámos um modelo de dados baseado em listas de adjacências bidirecionais locais. Sobre esta estrutura, programámos algoritmos específicos para realizar análises topológicas em plano secundário (detecção de inconsistências via BFS) e a simulação de caminhos heurística do *Smart Walker*.
- **Sistema de Internacionalização (i18next):** Integrámos a biblioteca **i18next** para gerir dicionários locais. Desenvolvemos o mapeamento que associa os dados tabulares da matriz de tradução do utilizador a ficheiros JSON de localização estáticos, carregados dinamicamente no navegador com o plugin de cliente `i18next-http-backend` sob o modelo *backendless*.

3.3.1 Justificação das Decisões de Design

Com base na análise comparativa das ferramentas existentes (ver Secção 2.4.7), as escolhas de design e de tecnologia para o PiaP foram fundamentadas nos seguintes pontos:

- **Fusão de Paradigmas (Visual vs. Textual):** Enquanto o Twine se destaca pela interface puramente visual e o Ink pelo controlo fino de estados lógicos complexos, o PiaP visa fundir ambas as forças, proporcionando um editor visual de grafos que não abdica da manipulação robusta de condicionais e variáveis (via macro-linguagem SugarCube).
- **Adoção do Formato Aberto Twee 3 e Limitação de Grafos:** A especificação aberta Tweak 3 possui um mapeamento direto entre passagens e grafos, o que

suporta a decisão de adotá-lo como base de persistência do PiaP. Contudo, para permitir que o sistema leia as saídas e as renderize visualmente como arestas de forma fiável, o projeto introduz um compromisso de design (*trade-off*) em relação ao Twine tradicional: não é suportada a utilização de variáveis como destinos dinâmicos de hiperligações (ex: `[[Ir para o local|$local]]`), exigindo que todos os destinos do grafo sejam estáticos (conforme detalhado na Secção 2.5.3).

- **Validação Automática e Segurança do Fluxo:** O ChoiceScript é valorizado pelas suas ferramentas robustas de teste automático (`randomtest`), uma lacuna crítica do Twine. O PiaP incorpora de raiz este princípio ao disponibilizar o validador estático (BFS) e o Smart Walker no simulador para preencher esta falta, garantindo a correção imediata do fluxo narrativo.
- **Autonomia e Licenciamento Livre:** A restrição de uso comercial imposta pelo ChoiceScript salienta a importância de desenvolver uma ferramenta que promova a autonomia dos autores, permitindo a exportação de projetos sob licenças livres e sem taxas adicionais.

3.4 Modelação de Dados: Lista de Adjacência

Para representar o fluxo da história no editor, o sistema utiliza uma **Lista de Adjacência Bidirecional**. Cada cena no modelo de dados armazena de forma independente dois conjuntos de caminhos:

- **Saídas (Sucessores):** Um vetor que contém as escolhas explícitas disponíveis para o leitor, mapeando o texto da opção e o ID do nó de destino.
- **Entradas (Predecessores):** Um vetor que regista as referências de todos os outros nós que contém escolhas a apontar para o nó atual.

Esta representação permite à aplicação realizar o mapeamento reverso instantâneo (*backward mapping*), alimentando os painéis “Vens de...” e “Vais para...” na interface. A estrutura de dados JSON representativa de um nó apresenta o seguinte formato simplificado:

```

1 {
2   "id": "cena_praca",
3   "titulo": "A Praça Central",
4   "conteudo": "O sol brilha no mercado central.",
5   "saidas": [
6     { "texto_escolha": "Entrar na taberna", "destino": "cena_taberna" }
7   ],
8   "entradas": ["cena_taberna"]
9 }
```

Note-se que este modelo foca-se exclusivamente na estrutura topológica do grafo. As variáveis globais e as condições lógicas SugarCube não são serializadas na lista de

adjacências, sendo integradas diretamente no corpo de texto (**conteúdo**) das passagens e interpretadas em tempo de execução pelo simulador.

3.5 Parser Síncrono e Especificação Twee 3

O ficheiro `tweeParser.js` (localizado em `/src/utis/tweeParser.js`) trata da sincronização bidirecional em tempo real entre o canvas do React Flow e o formato de texto Twee 3 (Interactive Fiction Technology Foundation, 2020).

Esta sincronização acontece de forma síncrona diretamente na memória do navegador. Isto significa que qualquer alteração de coordenadas, edição de texto ou criação de novas ligações no canvas é traduzida de imediato para a estrutura Twee 3 interna (e o oposto também se verifica), sem tempos de espera ou passos de compilação. Como a ferramenta funciona inteiramente no lado do cliente (*client-side*), este mapeamento apenas mantém o estado da sessão atualizado; a persistência física dos dados no disco rígido requer sempre que o utilizador exporte o ficheiro de forma explícita.

3.5.1 Importação e Conversão de Coordenadas de Zonas

Durante a leitura e escrita de ficheiros, o parser converte as coordenadas de posição absolutas guardadas no formato Twee 3 (explicadas na Secção 2.5.2) em coordenadas locais quando um nó é filho de uma Zona (explicado na Secção 3.6.1). O React Flow exige que os nós filhos tenham posições relativas ao canto superior esquerdo da zona mãe, calculadas por:

$$X_{\text{relativo}} = X_{\text{absoluto}} - X_{\text{zona}}$$

$$Y_{\text{relativo}} = Y_{\text{absoluto}} - Y_{\text{zona}}$$

No processo inverso de exportação para ficheiro `.twee`, o parser executa a conversão inversa para manter a compatibilidade com outros compiladores de Twine:

$$X_{\text{absoluto}} = X_{\text{zona}} + X_{\text{relativo}}$$

$$Y_{\text{absoluto}} = Y_{\text{zona}} + Y_{\text{relativo}}$$

3.5.2 Passagens Especiais

O parser reconhece passagens reservadas de metadados da especificação Twee 3:

- `:: StoryTitle`: Define o título do projeto.
- `:: StoryData`: Contém as configurações globais em formato JSON, incluindo o identificador único da história (`ifid`), o formato de compilação (`SugarCube`), o nó de arranque (`start`) e o nível de zoom.
- `:: StoryInit`: Nó especial não-narrativo onde são declaradas e inicializadas as variáveis lógicas de controlo do estado da simulação (as quais servem de memória do jogo, conforme introduzido na Secção 3.1).

3.5.3 Modelo SugarCube (Ligações e Macros)

As arestas são extraídas do texto narrativo de cada nó através de expressões regulares que identificam ligações no formato clássico Twee:

- **Arestas Simples:** `[[Cena2]]`
- **Arestas com Alias ou Direcionamento:** `[[Ir para a Taverna|cena_taberna]]` ou `[[Abrir porta->cena_interior]]`

No caso das macros de alteração de estado (ex: `<<set $moedas += 10>>`) e condicionais lógicas (ex: `<<if $tem_chave>> ... <</if>>`), o parser de importação não as compila nem as remove do texto. O parser executa duas operações:

1. **Preservação de Código:** Mantém as macros intactas no corpo textual da passagem, permitindo que sejam interpretadas e executadas reativamente pelo simulador (*PlayMode*) durante a execução do jogo (ver Secção 3.7.3).
2. **Extração de Variáveis para Indexação:** Identifica os nomes das variáveis (iniciadas por `$`) e regista-as no painel de gestão de variáveis, permitindo que o autor as rastreie ou renomeie de forma atómica em todo o projeto.

3.6 Interação Visual, Edição e Acessibilidade

3.6.1 Zonas de Agrupamento Visual

As **Zonas de Agrupamento Visual** (*ZoneNode*) consistem em contentores espaciais retangulares que agrupam logicamente nós de cena relacionados (como capítulos ou localizações), conforme ilustrado na Figura 3.2.

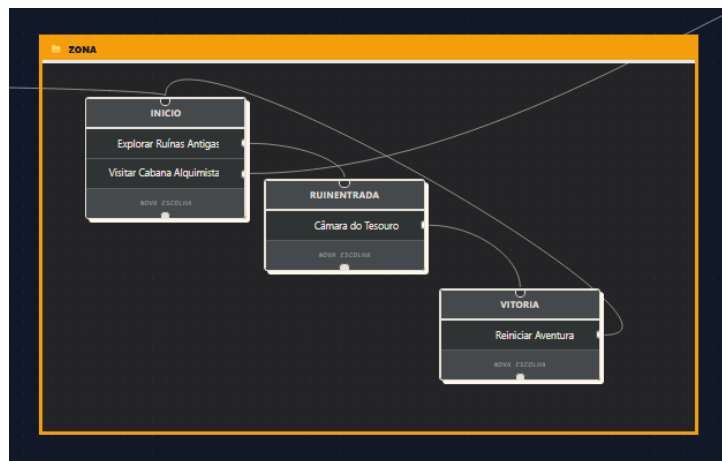


Figura 3.2: Exemplo de uma Zona de Agrupamento Visual com nós de cena filhos no canvas.

A sua implementação como nós de grupo personalizados exigiu contornar um problema de colisão de cliques no React Flow. Como o contentor cobre uma área retangular considerável sobreposta aos nós de cena interiores, os *pointer events* capturavam os

cliques no ecrã, impedindo a seleção ou arrasto dos filhos. Para resolver isto, definiu-se a propriedade CSS `pointer-events: none` dinamicamente na configuração global do React Flow para o contentor da Zona em `App.jsx`. Em paralelo, no componente `ZoneNode.jsx`, forçou-se `pointer-events: auto` exclusivamente nas pegas de redimensionamento (*NodeResizer*) e na barra de cabeçalho (utilizada para arrastar a zona em bloco com todos os seus filhos).

As Zonas suportam ainda imagens de fundo personalizadas (*backgrounds*) geridas através da propriedade reativa `bgImage` nos metadados do nó, a qual é decodificada em tempo de execução e aplicada via CSS.

3.6.2 Arestas Curvadas com Waypoints

Para evitar a sobreposição visual das ligações e o cruzamento cego sobre os nós de cena, a plataforma implementa arestas personalizadas que utilizam a interpolação por *Splines* Cúbicas de Catmull-Rom (Catmull et al., 1974). Esta lógica encontra-se implementada no componente `EditableEdge.jsx` (`/src/components/EditableEdge.jsx`), com o resultado visual exemplificado na Figura 3.3.



Figura 3.3: Exemplo de aresta suavizada com caminho curvado personalizável no canvas.

O autor pode criar novos marcadores (*waypoints*) com um duplo clique e arrastá-los livremente pelo ecrã. A curva é recalculada em tempo de execução à medida que os marcadores são movidos, interpolando a sua posição de forma contínua para manter a linha fluida e desimpedida de outros elementos do grafo.

3.6.3 Acessibilidade e Identidade Visual Customizada

Tendo em conta que o desenvolvimento do projeto beneficiou de validações diretas com utilizadores daltónicos, projetou-se uma estratégia de acessibilidade redundante baseada numa identidade visual neo-brutalista. A adoção desta estética neo-brutalista fundamentou-se em estudos que demonstram que contornos espessos, pesos tipográficos elevados e contrastes marcados beneficiam utilizadores com baixa acuidade visual ou daltonismo (Babich, 2023; Loranger, 2022). O aspeto geral desta interface e os seus elementos gráficos encontram-se representados na Figura 3.4.

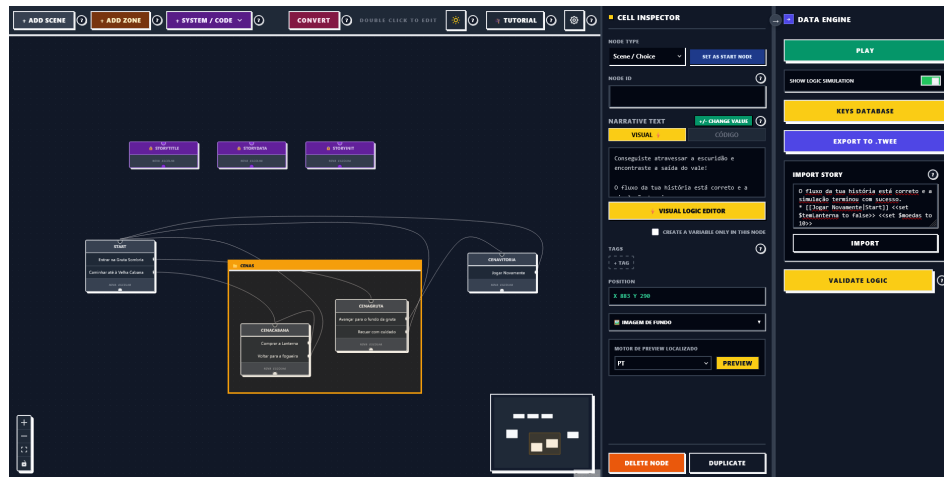


Figura 3.4: Interface gráfica global do editor (*Plot in a Pot*), ilustrando a identidade visual neo-brutalista de alto contraste com contornos espessos e sinalização redundante para acessibilidade.

O nosso trabalho de design e programação focou-se nos seguintes pilares:

- **Delimitação por Contornos Sólidos:** Implementação de contornos pretos de 2px (`border-2`) e sombras projetadas rígidas sem difusão (`shadow-[4px_4px_0px_#000]`) em todos os componentes visuais (*StoryNode*, *ZoneNode* e painéis), eliminando a dependência de cores ou gradientes suaves para distinguir elementos.
- **Diferenciação Estrutural de Arestas:** As transições normais no grafo são desenhadas com traçados contínuos, enquanto as ligações inacessíveis ou com erros lógicos são renderizadas com traçados tracejados (`strokeDasharray="5,5"`). Esta distinção de padrão garante que o utilizador consiga identificar caminhos quebrados sem depender exclusivamente da cor.
- **Sinalização Semântica Dupla de Alertas:** Configuração de etiquetas textuais explícitas (como o selo “Aviso” em fundo laranja nos nós com erros estruturais) em conjunto com listagens descritivas nos painéis, evitando a sinalização baseada apenas no par de cores verde/vermelho.

3.6.4 Modelação de Dados e Sincronização com o React Flow

Para além da representação abstrata da arquitetura, a implementação prática exige uma sincronização bidirecional em tempo real entre a interface gráfica gerida pela biblioteca *React Flow* e a estrutura de dados semântica interna da história.

No PiaP, o estado centralizado da narrativa é mantido na raiz da aplicação (`App.jsx`) e de forma modular através de ganchos personalizados (*custom hooks*), recorrendo a três vetores de dados estruturados em memória local:

1. **Vetor de Nós do React Flow (nodes):** Contém a representação posicional e de renderização de cada elemento no canvas. Cada objeto de nó segue a especificação rígida da biblioteca:

```
{
```

```

    id: "cena_taberna",
    type: "storyNode",
    position: { x: 250, y: 180 },
    data: {
      title: "Interior da Taberna",
      content: "O cheiro a hidromel preenche a sala...",
      tags: ["secreto"],
      bgImage: ""
    },
    width: 280,
    height: 150
  }

```

Se o nó representar uma Zona de Agrupamento (*ZoneNode*), o tipo é alterado para *"zoneNode"* e o objeto passa a incluir propriedades de agrupamento como *parentId* nos nós filhos e dimensões explícitas de largura e altura controladas pelo utilizador.

2. **Vetor de Arestas do React Flow (*edges*):** Modela a conectividade geométrica das escolhas narrativas. Cada aresta é configurada como um componente personalizado (*EditableEdge*) e armazena os pontos geométricos intermédios (*waypoints*) inseridos pelo autor para suavizar as curvas:

```

{
  id: "edge-cena_praca-cena_taberna",
  source: "cena_praca",
  target: "cena_taberna",
  type: "editableEdge",
  data: {
    choiceText: "Entrar na taberna",
    waypoints: [{ x: 120, y: 150 }, { x: 180, y: 160 }]
  }
}

```

3. **Lista de Adjacência Bidirecional (*adjacencyList*):** Representa o grafo direcionado de estados do ponto de vista do motor lógico. É recalculada de forma síncrona a cada modificação nos vetores de nós ou arestas. Esta lista mapeia cada identificador de cena ao seu conjunto de sucessores (saídas) e predecessores (entradas), permitindo ao validador e ao motor de simulação executar travessias e pesquisas rápidas.

Esta tripla modelação é sincronizada de forma atómica. Quando o autor arrasta um nó ou altera o texto de uma escolha no painel de detalhes, os manipuladores de eventos reativos do React Flow (como *onNodesChange* ou *onEdgesChange*) capturam a alteração física. Esta alteração é processada e encaminhada pelos ganchos personalizados

em `App.jsx`, gerando novos vetores imutáveis de estado que forçam a atualização dos componentes visuais e o recálculo imediato da lista de adjacências e do mapa de erros lógicos.

3.7 Validação Estática e Simulação

O motor de autoria do *Plot in a Pot* integra ferramentas robustas de validação e execução reativa. Para além de permitir a escrita de histórias, a plataforma disponibiliza um sistema de verificação lógica que audita a estrutura narrativa de forma estática e uma sandbox de simulação dinâmica dotada de agentes de navegação autónoma.

3.7.1 Algoritmo de Validação Estática e Exploração do Espaço de Estados (BFS)

O validador estático, implementado em `storyValidator.js` e `storyTraversal.js`, resolve o problema de acessibilidade e integridade da narrativa. Em grafos estáticos simples, uma travessia clássica de Pesquisa em Largura (BFS) seria suficiente para auditar o grafo. Contudo, em NI, as arestas (hiperligações) dependem frequentemente de expressões lógicas e variáveis de estado (macros `<<if>>` e `<<set>>` do SugarCube). A acessibilidade de um nó v não depende apenas do caminho geométrico, mas também da valoração atual das variáveis de estado (memória).

Assim, a validação lógica é modelada como um problema de *Exploração do Espaço de Estados* (*State Space Exploration*), onde o grafo de transições real é constituído por pares $\langle v, \sigma \rangle$, nos quais $v \in \mathcal{N}$ representa o nó do editor e $\sigma : \mathcal{V} \rightarrow \mathcal{D}$ representa um mapeamento de variáveis lógicas globais para os seus respetivos domínios de valores.

O algoritmo executa a travessia de acordo com os seguintes passos estruturados:

1. **Inicialização e Pré-computação:** Para garantir a eficiência temporal ($O(1)$ em lookups), o algoritmo pré-computa em memória:
 - Um mapa de nós $NodeMap : ID \rightarrow Node$, indexando todos os vértices da narrativa.
 - Um mapa de adjacências de arestas $EdgesBySource : ID \rightarrow List\langle Edge \rangle$.

O ponto de entrada v_0 é resolvido por $findStartNode(NodeMap)$, e o estado inicial σ_0 é lido a partir das declarações de variáveis do nó de sistema *StoryInit*.

2. **Estrutura de Dados e Fila:** Uma fila síncrona Q é instanciada e inicializada com o tuplo de arranque:

$$Q \leftarrow \text{enqueue}(Q, \langle v_0, \sigma_0, \text{path} = [v_0.\text{label}] \rangle)$$

São criados conjuntos vazios para registar nós alcançáveis ($\mathcal{N}_{reach}$), arestas alcançáveis ($\mathcal{E}_{reach}$), e tabelas hash de estados visitados por nó ($VisitedStates : ID \rightarrow Set\langle String \rangle$) para evitar caminhos redundantes e ciclos infinitos.

3. **Ciclo de Processamento Principal:** Enquanto Q não estiver vazia, remove-se o primeiro elemento:

$$\langle u, \sigma, \text{path} \rangle \leftarrow \text{dequeue}(Q)$$

Para o nó corrente u , efetua-se o seguinte processamento:

- (a) Adiciona-se o identificador de u a $\mathcal{N}_{reach}$.
 - (b) Executam-se as instruções de modificação de estado contidas no texto do nó (por exemplo, macros de atribuição $\langle\langle \text{set} \rangle\rangle$), produzindo um novo estado de memória $\sigma' \leftarrow \text{applyModifiers}(u.\text{content}, \sigma)$.
 - (c) Verifica-se se o estado σ' já foi visitado no nó u . Para tal, converte-se σ' numa representação string serializada $S_{\sigma'}$ e verifica-se se $S_{\sigma'} \in \text{VisitedStates}[u]$. Em caso afirmativo, o ramo é podado (*pruning*) para evitar ciclos redundantes no grafo.
 - (d) Para evitar a explosão do espaço de estados em loops que incrementam variáveis infinitamente, impõe-se um limite de segurança de $K = 10$ estados únicos por nó. Se $|\text{VisitedStates}[u]| \geq K$, o algoritmo emite um aviso de ciclo infinito no terminal e poda o caminho.
 - (e) Caso contrário, insere-se $S_{\sigma'}$ em $\text{VisitedStates}[u]$ e regista-se a rota percorrida e o estado final na tabela de histórico de chegada $\text{ArrivalHistory}[u]$.
4. **Avaliação de Transições e Condicionais:** Para cada aresta de saída $e \in \text{EdgesBySource}[u]$ que conecta o nó u ao nó de destino w :
- (a) O validador extrai a hiperligação textual associada à aresta e do conteúdo de u .
 - (b) Avalia-se a condição de acessibilidade associada a essa escolha através da função $\text{canAccessChoice}(u.\text{content}, \text{link}, \sigma')$. Esta função analisa se o link está envolto por alguma macro condicional $\langle\langle \text{if} \rangle\rangle$ e avalia a sua expressão lógica contra a memória σ' .
 - (c) Se a condição for satisfeita (*true*), adiciona-se o ID da aresta e a $\mathcal{E}_{reach}$ e enfileira-se o próximo estado:

$$Q \leftarrow \text{enqueue}(Q, \langle w, \sigma', \text{path} \cup [w.\text{label}] \rangle)$$

Após o término da travessia (quando a fila Q fica vazia), o módulo de auditoria em `storyValidator.js` pós-processa os resultados recolhidos para isolar as falhas estruturais, categorizando-as em:

- **Nós Órfãos** (V_{orphan}): Nós $v \in \mathcal{N}$ que não pertencem a $\mathcal{N}_{reach}$ e cujas etiquetas não incluem a tag `'secreto'`.
- **Becos sem Saída** (V_{dead_end}): Nós $v \in \mathcal{N}_{reach}$ que contêm hiperligações cujo destino aponta para um ID de nó inexistente no grafo ($w \notin \mathcal{N}$).
- **Arestas Inacessíveis** ($E_{unreachable}$): Arestas $e \in \mathcal{E}$ que não pertencem a $\mathcal{E}_{reach}$. Estas representam caminhos cujas condicionais lógicas nunca são satisfeitas sob nenhuma das valorações de variáveis exploradas.

3.7.2 Simulador em Tempo Real (PlayMode) e Smart Walker

O simulador de jogo (`PlayMode.jsx`) mimetiza o comportamento do interpretador SugarCube num ambiente controlado pelo navegador do autor. Para permitir a depuração em tempo real sem afetar os dados de edição, o motor executa sob um isolamento de contextos:

- **Separação de Estilos:** Os nós categorizados com a tag `css` têm o seu conteúdo concatenado em tempo de execução e injetado sob a forma de uma tag `<style id='playmode-styles'>` na raiz do cabeçalho da página, sendo totalmente limpos ao sair da simulação.
- **Isolamento de Scripts:** Os nós lógicos contendo JavaScript são processados como funções anónimas com escopo isolado através do construtor `new Function('state', 'State', 'setup', script_content)`, impedindo que variáveis globais escritas pelo utilizador interfiram com os estados internos do React ou do canvas do React Flow.

Para testes automáticos de caminhos narrativos, a plataforma disponibiliza o agente autónomo **Smart Walker** (implementado em `storyTraversal.js`). Em vez de selecionar opções de forma puramente pseudo-aleatória (o que levaria a uma baixa cobertura de caminhos e retenção em ciclos curtos), o agente utiliza uma heurística baseada em *Procura por Novidade* (*Novelty Search*).

Dada a vizinhança do nó atual v , correspondente ao conjunto de nós adjacentes alcançáveis $N(v)$, o agente consulta o histórico de visitas acumuladas do simulador. Para cada opção de transição que leva ao nó vizinho $w \in N(v)$, o Smart Walker obtém o número acumulado de visitas históricas $H(w)$ registado na memória global do simulador. O Walker seleciona a transição para o nó vizinho w_{next} que minimiza a frequência de visitas:

$$w_{next} = \arg \min_{w \in N(v)} H(w)$$

Este comportamento força o simulador a procurar caminhos narrativos raros e ramos profundos da história de forma automatizada, conseguindo mapear e encontrar becos sem saída ou ciclos lógicos fechados numa fração de segundo sem intervenção manual do autor.

3.7.3 Motor de Compilação JIT de Macros SugarCube

Para suportar as avaliações e atribuições de variáveis de estado descritas, a aplicação possui um compilador *Just-In-Time* (JIT) simplificado estruturado em `logicParser.js` e `sugarcubeLogic.js`:

- **Análise Léxica e Mapeamento:** O interpretador analisa o conteúdo textual dos nós através de varrimentos que procuram padrões de macros `<<set>>` e `<<if>>`. Traduz os operadores clássicos da linguagem SugarCube para os operadores lógicos válidos de JavaScript, conforme resumido na Tabela 3.1.

- **Árvore de Sintaxe Abstrata (Árvore de Sintaxe Abstrata (Abstract Syntax Tree) (AST)):** O interpretador de condicionais analisa blocos lógicos aninhados construindo uma AST simples que reflete a precedência de cláusulas. Esta estrutura é então avaliada em tempo de execução para calcular o valor de verdade das expressões.

Tabela 3.1: Mapeamento de operadores lógicos SugarCube para JavaScript.

Operador SugarCube	Operador JavaScript	Significado
is / eq	===	Igualdade estrita
neq	!==	Diferença estrita
and	&&	E lógico
or		OU lógico
to	=	Atribuição de valor
\$variavel	state.variavel	Acesso ao estado reativo

3.8 Integração de Modelos de Linguagem (LLMs)

Para além do núcleo lógico de grafos e da simulação estática, a plataforma disponibiliza um assistente baseado em Modelos de Linguagem de Grande Escala (LLMs) para facilitar a importação e co-criação de histórias. A conceção e implementação desta funcionalidade assenta nos seguintes aspetos:

3.8.1 Arquitetura de Integração Local

A integração com inteligência artificial foca-se em fornecer uma interface de comunicação local e assíncrona entre o editor gráfico e o servidor de inferência do utilizador. O trabalho de desenvolvimento realizado pela equipa consistiu em:

- **Módulo de Comunicação REST (aiGenerator.js):** Programámos o cliente HTTP em Javascript para estabelecer a ligação assíncrona com o endpoint local do Ollama (<http://localhost:11434>), enviando os cabeçalhos e o corpo JSON com os parâmetros de temperatura e tokens.
- **Engenharia de Prompt de Conversão:** Desenvolvemos e validámos um prompt de sistema especializado, com regras sintáticas rígidas, concebido para forçar o modelo de linguagem a gerar respostas estritamente em conformidade com a especificação Tweep 3 (cabeçalhos, metadados e arestas literais), mitigando alucinações.
- **Importação Automatizada Pós-Resposta:** Desenvolvemos o fluxo de importação que recebe o texto estruturado retornado pela API local, encaminha-o para o nosso parser e gera automaticamente os nós e arestas correspondentes no canvas.

A decisão de basear este assistente em inferência local (via Ollama) assenta nos seguintes compromissos:

- **Privacidade e Soberania dos Dados:** Ao executar o modelo localmente no computador do autor, garante-se que toda a propriedade intelectual da história e os textos criados permanecem estritamente confidenciais. Não há partilha de dados com servidores externos de terceiros nem risco de os textos literários serem recolhidos para treino de modelos comerciais.
- **Custo Financeiro Zero:** APIs na nuvem implicam faturação contínua baseada em consumo de tokens, o que criaria barreiras ao uso regular. A inferência local permite a utilização gratuita e ilimitada da funcionalidade de conversão inteligente.

3.8.2 Papel do LLM: Conversão e Importação Automática de Histórias

A nossa abordagem de design delimitou de forma clara o papel do modelo de linguagem no ciclo de vida da narrativa. Visto que os LLMs tendem a alucinar, criar caminhos impossíveis e ignorar restrições de ciclos, o modelo não é integrado dinamicamente na simulação direta do jogo.

O papel do LLM foi desenhado por nós para atuar estritamente como uma ferramenta de conversão sintática em fase de pré-processamento. O modelo lê a história linear fornecida pelo autor e converte-a para a norma Tweep 3. A partir desse ponto, o nosso motor lógico e o validador BFS assumem a responsabilidade exclusiva de renderizar o canvas visual e garantir a integridade dos caminhos e das variáveis de estado no editor.

4

Implementação e Avaliação

Este capítulo detalha o processo de implementação prática do *Plot in a Pot*, descrevendo a estrutura física do código, detalhes de componentes e utilitários, testes lógicos de inteligência artificial, benchmarks de desempenho e a metodologia e resultados dos testes de utilizador realizados para validar o sistema.

4.1 Estrutura do Código e Detalhes de Implementação

Para além da fundamentação teórica, o processo de desenvolvimento envolveu a estruturação de uma base de código robusta, modular e altamente reativa. O software está organizado de forma a garantir a separação de conceitos (*Separation of Concerns*), dividindo-se entre componentes de interface visuais, ganchos customizados (*custom hooks*) e utilitários de processamento lógico puro.

4.1.1 Gestão de Estado Global, Reatividade e Histórico

O componente principal `App.jsx` atua como o orquestrador reativo central do sistema. Para evitar a criação de um ficheiro monolítico, a equipa encapsulou a lógica de negócio em ganchos customizados (*custom hooks*) específicos:

- **useSettings:** Centraliza o estado e a persistência em *LocalStorage* das opções de visualização (ex: exibição de erros, nós secretos, e grau de desfocagem da imagem de fundo).
- **useUndoRedo:** Controla as pilhas de estados anteriores (*past*) e futuros (*future*) para as operações atômicas de desfazer e refazer no canvas.
- **useChoiceSync:** Implementa a análise sintática em tempo real para sincronizar as ligações textuais com as arestas correspondentes a escolhas.
- **useNodeOperations:** Encapsula a criação, remoção, duplicação e atualização de propriedades semânticas e posicionais de nós e zonas no editor.
- **useGraphHandlers:** Gere as interações gráficas ligadas ao canvas (ligar nós, atualizar arestas, gerir arrasto e inserção em zonas de agrupamento).

- **useKeyboardShortcuts:** Escuta eventos do teclado para acionar atalhos de produtividade global.
- **useProjectPersistence:** Controla a gravação automática periódica, tradução em tempo real, importação bidirecional de ficheiros Tweepy e carregamento de modelos base.

A implementação do histórico colocou um desafio relevante no contexto de reatividade do React. Como o React Flow atualiza objetos de nós e arestas por referência, a simples inserção de estados na pilha de histórico resultaria na mutação dos estados passados sempre que o utilizador arrastasse um nó. Para resolver esta limitação, implementou-se uma estratégia de clonagem profunda (*deep clone*) em cada instantâneo, conforme demonstrado na Listagem 1.

```

1 // Registo de snapshots e funcoes de controlo em useUndoRedo.js
2 const takeSnapshot = useCallback(() => {
3   setPast(prev => [
4     ...prev.slice(-50),
5     {
6       nodes: JSON.parse(JSON.stringify(nodesRef.current)),
7       edges: JSON.parse(JSON.stringify(edgesRef.current))
8     }
9   ]);
10  setFuture([]);
11 }, []);
12
13 const undo = useCallback(() => {
14   if (past.length === 0) return;
15   const previous = past[past.length - 1];
16   setFuture(prev => [
17     {
18       nodes: JSON.parse(JSON.stringify(nodesRef.current)),
19       edges: JSON.parse(JSON.stringify(edgesRef.current))
20     },
21     ...prev
22   ]);
23   setPast(prev => prev.slice(0, -1));
24   setNodes(previous.nodes);
25   setEdges(previous.edges);
26 }, [past]);

```

Listagem 1: Mecanismo de clonagem profunda e gestão da pilha de histórico encapsulado em *useUndoRedo.js*.

Esta estrutura suporta o fluxo reativo unidirecional da aplicação. Quando o autor edita o texto de uma passagem no painel de propriedades, um evento de alteração é propagado para o estado central em `App.jsx`, que força a reavaliação síncrona e em

segundo plano de dois subsistemas independentes: o parser de texto (que extrai as novas arestas baseadas em ligações double-bracket e reconstrói a Lista de Adjacência em memória) e o validador estático (que executa a travessia BFS sobre a nova lista de adjacência para detetar inconsistências e desenhar alertas visuais de aviso diretamente no canvas).

4.1.2 Componentes Gráficos e Desafios de Interação (Canvas, Nós e Arestas)

Os nós gráficos que habitam o canvas estendem os componentes base da biblioteca React Flow (conforme ilustrado na Figura 4.1):

- **StoryNode.jsx:** Renderiza o título da passagem, o extrato de texto da história e as portas de ligação (*handles*) de entrada e saída.
- **ZoneNode.jsx (Agrupamento Visual):** Implementa o agrupamento de capítulos. Para resolver o conflito de eventos de rato (onde a Zona sobreposta impedia cliques no canvas de fundo ou nos nós filhos), aplicou-se a propriedade CSS `pointer-events: none` ao contêiner geral da Zona, e ativou-se `pointer-events: all` exclusivamente para as bordas de redimensionamento e barra de cabeçalho, como demonstrado na Listagem 2.
- **EditableEdge.jsx (Arestas Curvas):** Implementa o traçado curvo interativo. O utilizador pode efetuar um duplo clique sobre a linha para introduzir um ponto de controlo (*waypoint*) geométrico. O algoritmo (Listagem 3) interpola os pontos e converte a spline Catmull-Rom em comandos de curva Bézier cúbica SVG (C) compatíveis com o navegador.

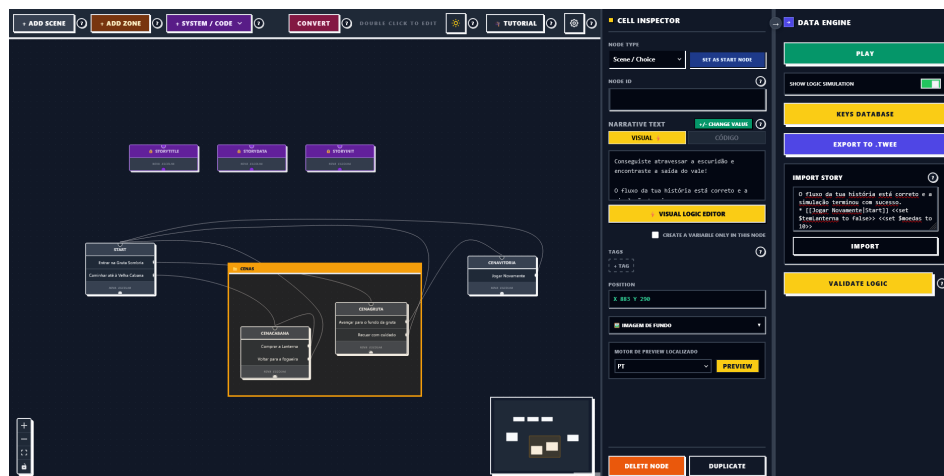


Figura 4.1: Interface gráfica do canvas de grafos do Plot in a Pot, demonstrando a estética neo-brutalista de alto contraste com nós de cena e agrupamentos visuais (Zonas).

```

1 <!-- Estrutura simplificada do componente ZoneNode -->
2 <div
3   className="zone-node-container"
4   style={{ pointerEvents: 'none' }}
5 >
6   <div
7     className="zone-node-header"
8     style={{ pointerEvents: 'all' }}
9   >
10    <h3>{data.title}</h3>
11  </div>
12  <div
13    className="zone-node-body"
14    style={{ border: '2px dashed #000' }}
15  >
16    { /* Os nos filhos sao renderizados no topo e respondem a cliques */ }
17  </div>
18 </div>

```

Listagem 2: Resolução de eventos de rato através de regras CSS em *ZoneNode.jsx*.

```

1 // Geracao da curva SVG com interpolacao de pontos geometricos
2 export function getCatmullRomPath(points) {
3   if (points.length < 2) return '';
4   let path = `M ${points[0].x} ${points[0].y}`;
5   for (let i = 0; i < points.length - 1; i++) {
6     const p0 = points[i - 1] || points[i];
7     const p1 = points[i];
8     const p2 = points[i + 1];
9     const p3 = points[i + 2] || p2;
10
11     // Calculo dos pontos de controlo Bezier baseados em Catmull-Rom
12     const cp1x = p1.x + (p2.x - p0.x) / 6;
13     const cp1y = p1.y + (p2.y - p0.y) / 6;
14     const cp2x = p2.x - (p3.x - p1.x) / 6;
15     const cp2y = p2.y - (p3.y - p1.y) / 6;
16
17     path += ` C ${cp1x} ${cp1y}, ${cp2x} ${cp2y}, ${p2.x} ${p2.y}`;
18   }
19   return path;
20 }

```

Listagem 3: Algoritmo de interpolação e desenho de curvas no componente *EditableEdge.jsx*.

4.1.3 Mapeamento Reverso, Expressões Regulares e Matriz de Localização

Para gerir a complexidade de narrativas extensas, a equipa implementou as seguintes soluções:

- **Mapeamento Reverso no Inspector:** O painel lateral `Inspector.jsx` lê e sincroniza os dados do nó selecionado. Para evitar que o autor tenha de inspecionar visualmente todo o canvas, o componente realiza um mapeamento reverso sobre a lista de adjacências para identificar e expor em tempo real todos os caminhos incidentes de entrada e saída (Listagem 4).
- **Refatorização de Variáveis em VariablesModal:** Quando o utilizador renomeia uma variável global, o sistema varre e atualiza de forma atômica todas as ocorrências em todos os nós da história. A substituição recorre a uma expressão regular compilada com barreira de palavras (`\b`) para evitar sobreposição de nomes parciais (conforme a Figura 4.2 e Listagem 5).
- **Matriz de Localização e Parser CSV:** O painel `TranslationMatrix.jsx` expõe uma grelha de tradução (ilustrada na Figura 4.3). Para permitir a interoperabilidade com editores de folhas de cálculo externos, desenvolveu-se um parser de importação CSV personalizado aderente à norma RFC 4180 (Listagem 6).

```
1 // Calculo de caminhos incidentes no painel lateral
2 const getConnections = (nodeId, edges, nodes) => {
3   const incoming = edges
4     .filter(edge => edge.target === nodeId)
5     .map(edge => {
6       const srcNode = nodes.find(n => n.id === edge.source);
7       return { id: edge.source, label: srcNode?.data?.label || edge.source };
8     });
9
10  const outgoing = edges
11    .filter(edge => edge.source === nodeId)
12    .map(edge => {
13      const tgtNode = nodes.find(n => n.id === edge.target);
14      return { id: edge.target, label: tgtNode?.data?.label || edge.target };
15    });
16
17  return { incoming, outgoing };
18 };
```

Listagem 4: Mapeamento dinâmico de nós predecessores e sucessores em `Inspector.jsx`.

```
1 // Substituicao em lote de variaveis nos textos dos nos
2 const refactorVariableInNodes = (oldName, newName, currentNodes) => {
3   const escapedOld = oldName.replace(/[-\/\\^$*+?.()|[\]{}]/g, '\\$&');
4   const regex = new RegExp(escapedOld + '\\b', 'g');
5
6   return currentNodes.map(node => {
7     const text = node.data.content || '';
8     if (regex.test(text)) {
9       return {
10         ...node,
11         data: { ...node.data, content: text.replace(regex, newName) }
12       };
13     }
14     return node;
15   });
16 };
```

Listagem 5: Refatorização automática de variáveis lógicas baseada em Regex no *VariablesModal.jsx*.

```
1 // Conversao de texto CSV bruto numa matriz de traducao
2 const parseCSVToTranslations = (csvText) => {
3   const rows = [];
4   let currentRow = [];
5   let currentCell = '';
6   let inQuotes = false;
7
8   for (let i = 0; i < csvText.length; i++) {
9     const char = csvText[i];
10    const nextChar = csvText[i + 1];
11
12    if (char === '"') {
13      if (inQuotes && nextChar === '"') {
14        currentCell += '"';
15        i++;
16      } else {
17        inQuotes = !inQuotes;
18      }
19    } else if (char === ',' && !inQuotes) {
20      currentRow.push(currentCell.trim());
21      currentCell = '';
22    } else if ((char === '\n' || char === '\r') && !inQuotes) {
23      if (char === '\r' && nextChar === '\n') i++;
24      currentRow.push(currentCell.trim());
25      rows.push(currentRow);
26      currentRow = [];
27      currentCell = '';
28    } else {
29      currentCell += char;
30    }
31  }
32  return rows;
33 };
```

Listagem 6: *Parser de importação de ficheiros CSV com suporte à norma RFC 4180 em TranslationMatrix.jsx.*

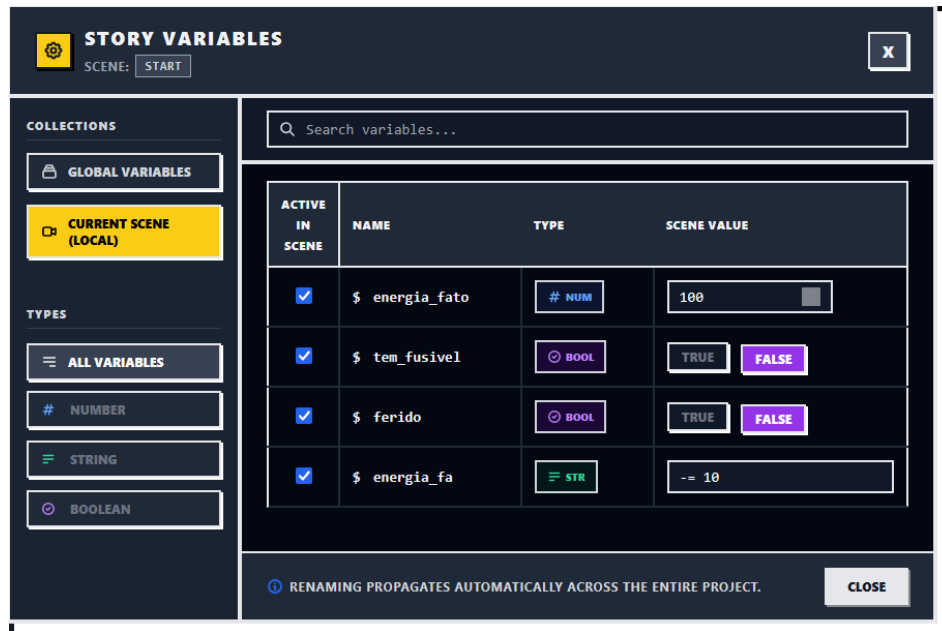


Figura 4.2: Diálogo de gestão de variáveis globais (*VariablesModal*), apresentando os tokens de dados e a renomeação segura baseada em *Regex*.

LOCALIZATION MATRIX			
KEY	PT (source)	EN	ACT
story_intro	acordei à entrada de um valejão gelado. sentei os bolsos pesados. À tua frente, um portão de ferro.	You wake up at the entrance of a cold valley. Your pockets feel heavy. In front of you, a gate.	+
story_merchant	Um mercador sombrio sorri para ti por trás do seu balcão de madeira. "Procure as entradas para eu...	A shady merchant smiles at you from behind his wooden counter. "Looking for a way out?"	+
story_gate	O portão de ferro é gelado ao toque. Mesmo quando o tocas, mesmo quando o tocas, mesmo quando o tocas...	The iron gate is ice-cold to the touch. Even when you touch it, even when you touch it, even when you touch it...	+
story_locked	O portão abre-se com um rangido. Passa para o interior da gruta de cristal luminosa...	The gate creaks open. You step into the glowing crystal cavern. You successfully bypassed the...	+
choices_return_gate	Volta para o portão de ferro	Head back to the iron gate	+
choices_return_merch	Volta ao mercador	Go back to the merchant	+
choices_try_open	Inserir a chave de madeira na fechadura	Insert the wooden key into the lock	+

Figura 4.3: Matriz de localização integrada no editor, exibindo a grelha de chaves de texto e a respetiva tradução direta para os idiomas selecionados.

4.1.4 Sandbox de Simulação, Edição Lógica Estruturada e Utilitários

Para além dos componentes gráficos, a equipa implementou módulos de simulação, edição estruturada de lógica e processamento sintático puro (que operam fora do ciclo de renderização direta do DOM):

- **PlayMode.jsx (Sandbox Isolada):** Mimetiza a execução da história. Para evitar que variáveis de script globais escritas pelo utilizador interfiram com os estados internos da aplicação React, o motor executa os blocos dinâmicos sob uma sandbox isolada baseada no construtor de funções dinâmicas (como exibido na Figura 4.4 e Listagem 7).

- **VisualBlocksEditor.jsx (Edição Estruturada):** Permite que autores configurem prosa, setters de variáveis e escolhas condicionais graficamente, as quais são convertidas em macros SugarCube (apresentada na Figura 4.5 e Listagem 8).
- **tweeParser.js (Parser de Layout):** Converte o canvas reativo em ficheiro de texto Twee 3. O parser processa as coordenadas posicionais, convertendo-as de absolutas para relativas caso os nós estejam contidos em Zonas de Agrupamento (Listagem 9).
- **storyTraversal.js e storyValidator.js (Validador BFS):** Executa a travessia de espaço de estados. A Listagem 10 ilustra o ciclo principal do algoritmo, evidenciando a paragem reguladora de segurança de visitas a nós a um limite $K = 10$ estados lógicos distintos.
- **aiGenerator.js (Assistente de Conversão com IA):** Efetua chamadas assíncronas HTTP REST para o servidor Ollama na porta padrão, estruturando prompts de sistema para desativar saídas coloquiais e garantir o formato Twee 3 (Listagem 11).

```

1 // Execucao controlada de scripts do utilizador
2 const executeScriptInSandbox = (scriptContent, gameState) => {
3   try {
4     const sandbox = new Function('state', `
5       with(state) {
6         ${scriptContent}
7       }
8     `);
9     const stateClone = JSON.parse(JSON.stringify(gameState));
10    sandbox(stateClone);
11    return stateClone;
12  } catch (error) {
13    console.error("Erro na execucao do script:", error);
14    return gameState;
15  }
16 };

```

Listagem 7: Mecanismo de execução de scripts de lógica com isolamento de contexto no *Play-Mode.jsx*.

```

1 // Conversao de logica estruturada em texto de macros SugarCube
2 export function serializeLogicToText(logic) {
3   const parts = [];
4   if (logic.narrative) {
5     parts.push(logic.narrative);
6     parts.push('');
7   }
8   if (logic.setters && logic.setters.length > 0) {
9     logic.setters.forEach(s => {
10       if (s.variable.trim()) {
11         parts.push(`<<set ${s.variable.trim()} to ${s.value}>>`);
12       }
13     });
14     parts.push('');
15   }
16   return parts.join('\n');
17 }

```

Listagem 8: *Serialização da estrutura de formulários lógicos em macros SugarCube (logicParser.js).*

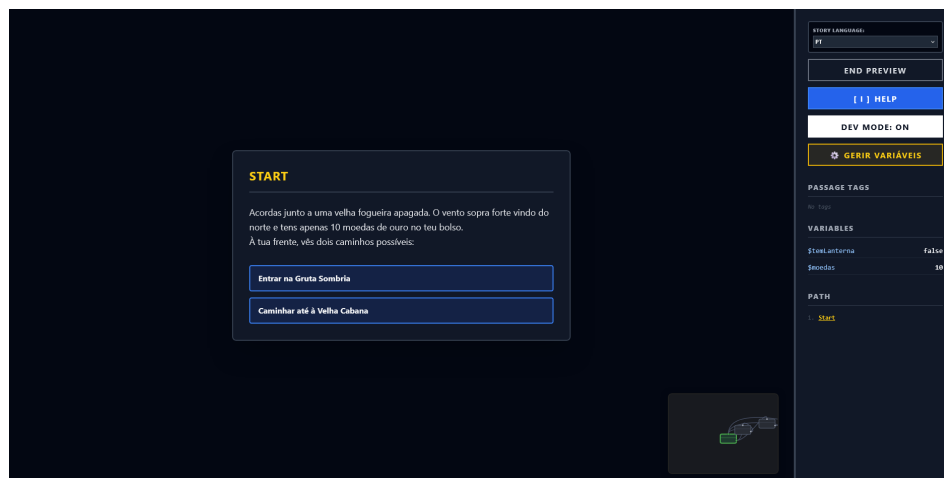


Figura 4.4: *Ecrã da sandbox de simulação (PlayMode), ilustrando a execução em tempo real da história com o painel de depuração de variáveis lógicas.*

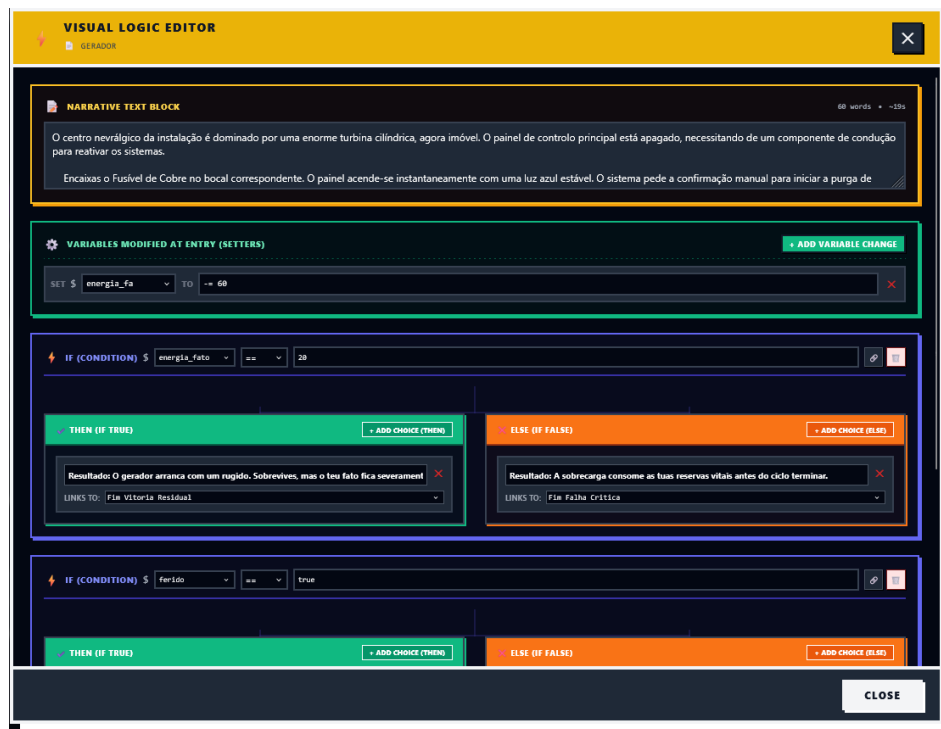


Figura 4.5: Interface do editor visual de lógica estruturada (VisualBlocksEditor), dividindo as passagens em painéis de prosa, setters de variáveis e escolhas condicionais.

```

1 // Conversao de coordenadas na leitura de passagens Twee 3
2 export function parsePassageMetadata(metaString, zoneMap) {
3   try {
4     const meta = JSON.parse(metaString || '{}');
5     let x = meta.position?.x ?? 0;
6     let y = meta.position?.y ?? 0;
7     const parentId = meta.parentId;
8
9     if (parentId && zoneMap.has(parentId)) {
10      const zone = zoneMap.get(parentId);
11      x = x - (zone.position?.x ?? 0);
12      y = y - (zone.position?.y ?? 0);
13    }
14    return { x, y, parentId, width: meta.size?.width, height: meta.size?.height };
15  } catch (e) {
16    return { x: 0, y: 0 };
17  }
18 }

```

Listagem 9: Processamento de coordenadas absolutas e relativas de nós e Zonas em *tweeParser.js*.

```

1 // Travessia e controlo de ciclos por limite de estados visitados
2 while (queue.length > 0) {
3   const { nodeId, state, path } = queue.shift();
4   const currentNode = nodeMap.get(nodeId);
5   if (!currentNode) continue;
6
7   const newState = applyModifiers(currentNode.data.content, state);
8   const stateStr = JSON.stringify(newState);
9
10  if (!visitedStates.has(nodeId)) visitedStates.set(nodeId, []);
11  if (visitedStates.get(nodeId).includes(stateStr)) continue;
12
13  if (visitedStates.get(nodeId).length >= 10) {
14    console.warn(`Ciclo infinito provavel no no ${currentNode.data.label}`);
15    continue;
16  }
17
18  visitedStates.get(nodeId).push(stateStr);
19  arrivalHistory.get(nodeId).push({ path, state: newState });
20 }

```

Listagem 10: *Ciclo principal de exploração do espaço de estados no motor storyTraversal.js.*

```

1 // Envio do prompt de conversao estrutural para o Ollama local
2 export async function generateTweeFromText(userText, modelName = 'llama3') {
3   const response = await fetch('http://localhost:11434/api/chat', {
4     method: 'POST',
5     headers: { 'Content-Type': 'application/json' },
6     body: JSON.stringify({
7       model: modelName,
8       messages: [
9         { role: 'system', content: 'You are a parser. Convert the story to Tweep
10           3. Output ONLY raw Tweep 3 format.' },
11         { role: 'user', content: userText }
12       ],
13       stream: false
14     })
15   });
16   const data = await response.json();
17   return data.message?.content || '';
18 }

```

Listagem 11: *Estrutura de comunicação assíncrona com o Ollama local em aiGenerator.js.*

4.2 Testes de Conversão e Comparação de Modelos de Linguagem (IA Local)

Para analisar a viabilidade do assistente de importação de texto livre, foram realizados testes com diferentes modelos de linguagem (Modelo de Linguagem de Grande Escala (Large Language Model)s (LLMs)) executados localmente. A escolha do ecossistema Ollama para estes testes justifica-se por ser uma ferramenta leve e otimizada de orquestração local que expõe uma API REST padrão, facilitando a integração direta com o editor e assegurando a soberania dos dados sem custos de operação.

4.2.1 Primeira Fase: Testes em Máquina Local (RTX 3050)

A primeira série de testes foi realizada a 9 de maio no computador do programador, equipado com uma placa gráfica Nvidia RTX 3050 (Laptop) com 4 GB de VRAM. O objetivo do teste consistiu em submeter uma narrativa linear escrita em linguagem natural para avaliar a capacidade do modelo em estruturar as passagens e os respetivos caminhos de ligação conformes à sintaxe Tweep 3. Selecionaram-se os seguintes modelos locais da biblioteca Ollama:

- Gemma 2B: ID 8ccf136fdd52
- Mistral 7B: ID 6577803aa9a0
- Qwen 3.5 9B: ID 6488c96fa5fa
- DeepSeek Coder 6.7B: ID ce298d984115

Além destes, foi testado o modelo Claude (acessível via API externa). Contudo, devido às restrições severas de memória da placa gráfica local de 4 GB, nenhum dos modelos locais funcionou adequadamente, gerando lentidão extrema no sistema operativo e erros de falta de memória (*out of memory*).

4.2.2 Segunda Fase: Testes com Maior Capacidade de Processamento

Para viabilizar a execução e avaliação comparativa dos modelos locais, a 19 de maio realizaram-se os mesmos testes num servidor de desenvolvimento equipado com uma GPU Nvidia Titan Xp com 12 GB de VRAM. A análise das respostas obtidas revelou falhas sistemáticas em cada modelo que inviabilizaram a importação direta:

- **Mistral 7B (`mistral_7b`):** O resultado revelou-se inadequado para a aplicação. O modelo separou quase todas as frases do texto em passagens independentes, gerando dezenas de pequenos nós para uma única cena, o que corrompeu o fluxo e a legibilidade do grafo. Adicionalmente, inseriu barras invertidas no final das linhas e lixo de terminal (como sequências de controlo ANSI). Posteriormente, identificou-se que esta poluição de caracteres se deveu ao modo como a ferramenta de linha de comandos (CLI) do Ollama lida com saídas direcionadas em processos automáticos sem terminal interativo (TTY), problema este contornável através da utilização de uma API REST ou via scripts Python.

- **Gemma 4B / 2B (gemma4_e4b):** O modelo esboçou a criação de passagens, mas inseriu os cabeçalhos de definição de passagem (o marcador `:`) no interior do corpo de texto de outros nós. Sendo que a especificação Tweep 3 proíbe passagens aninhadas, o parser interpretou a totalidade da resposta como um único nó corrompido.
- **Qwen 3 8B (qwen3_8b):** O resultado revelou-se inutilizável pois o modelo incluiu os seus blocos de raciocínio interno (prefixados por *“Thinking...”*) no início da resposta, inviabilizando o parsing. Adicionalmente, gerou um bloco JSON incompleto e caracteres ANSI no fluxo de texto.
- **Qwen 3.5 9B (qwen3.5_9b):** Apresentou o melhor desempenho de mapeamento e estrutura formal de caminhos, mas ainda assim falhou na conformidade total devido à inclusão de textos de raciocínio intermediário no início da resposta e à geração de nós órfãos desconetados da árvore principal.

4.2.3 Identificação do Bug Técnico no Ollama

Mais tarde, identificou-se que a presença de lixo do terminal (códigos ANSI, sequências [3D [K, etc.) se deve a um comportamento documentado do CLI do Ollama (Issue #14571 no GitHub). Quando o utilitário `ollama run` é invocado programaticamente a partir de um processo em segundo plano, ele falha em detetar a ausência de um terminal interativo (TTY), continuando a injetar caracteres de escape ANSI de controlo de cursor na resposta textual.

Para mitigar este problema no PiaP, identificaram-se três abordagens corretivas:

1. **Limpeza com Expressões Regulares:** Filtrar o texto de saída na aplicação aplicando uma expressão regular para expurgar as sequências ANSI.
2. **Descartar a Saída de Erros:** Redirecionar o fluxo de erro padrão (`stderr`) do processo do Ollama para o nulo.
3. **Uso da API REST do Ollama (Implementada):** Ligar a aplicação diretamente à API HTTP REST local do Ollama (na porta 11434). A API REST contorna o CLI oficial e devolve o texto limpo de formatação.

4.2.4 Estado Atual da Funcionalidade (Beta) e Trabalho Futuro

Conclui-se que o modelo Qwen 3.5 9B apresenta a melhor estrutura narrativa e conformidade com o parser. No entanto, mesmo com acesso a hardware dedicado com capacidade superior a 8 GB de VRAM, a funcionalidade de conversão por IA local permanece em fase experimental (beta), uma vez que a precisão de mapeamento estrutural e a correta filtragem de blocos de raciocínio interno (como tags `<think>...</think>`) exigem novos refinamentos de prompting e de filtros prévios de processamento. Esta funcionalidade constitui uma linha de investigação ativa para futuros desenvolvimentos.

4.3 Análise de Desempenho e Complexidade Algorítmica

Para validar a conformidade da aplicação com o requisito não-funcional de fluidez e desempenho (RNF03), que estipula latências de interação inferiores a 100 milissegundos, realizou-se uma avaliação da complexidade teórica e do comportamento prático do software sob diferentes cargas de dados.

4.3.1 Complexidade Algorítmica Teórica

Os módulos de cálculo do *Plot in a Pot* operam de forma síncrona no lado do cliente. A Tabela 4.1 resume a complexidade temporal e espacial assintótica (na notação Big- O) dos três algoritmos mais exigentes em termos computacionais.

Tabela 4.1: Complexidade algorítmica teórica dos subsistemas nucleares.

Subsistema / Algoritmo	Complexidade Temporal	Complexidade Espacial
Parser Bidirecional Twee 3	$O(L)$	$O(V + E)$
Validador de Espaço de Estados (BFS)	$O(V + E \cdot K)$	$O(V \cdot K + E)$
Interpolação de Arestas (Catmull-Rom)	$O(W)$	$O(W)$

Nota: L representa o número total de caracteres no ficheiro Twee; V e E indicam os vértices e arestas no grafo; W é o número de waypoints da aresta; K representa o limite regulador de estados visitados por nó ($K = 10$).

O parser opera com complexidade temporal linear $O(L)$ na leitura do texto completo. O validador BFS especial explora o espaço de estados com limite regulador K , garantindo complexidade controlada mesmo em grafos com ciclos infinitos, uma vez que a paragem é forçada no pior cenário após K visitas redundantes ao mesmo vértice. O motor de splines Catmull-Rom opera de forma linear com base no número de waypoints da aresta (W), processando apenas as curvas ativas visíveis no ecrã.

4.3.2 Benchmarks de Desempenho Empíricos

Para medir a velocidade real de execução das rotinas da aplicação, montou-se um ambiente de testes sintéticos simulando três escalas crescentes de projetos de autoria. Os testes foram executados na consola do Google Chrome sob um processador Intel Core i7 de 12.^a geração com 16 GB de RAM. Os tempos médios obtidos após 50 execuções sucessivas encontram-se sumarizados na Tabela 4.2.

Tabela 4.2: Resultados empíricos de benchmarks de desempenho (em milissegundos).

Escala do Projeto	Parsing Twee 3	Validação Lógica (BFS)	Sincronização Canvas (React Flow)
Pequeno (10 nós / 15 arestas)	2.1 ms	4.3 ms	12.5 ms
Médio (50 nós / 80 arestas)	8.4 ms	12.8 ms	28.1 ms
Grande (150 nós / 240 arestas)	21.6 ms	34.2 ms	62.4 ms

Nota: Os valores medem o tempo de CPU decorrido desde o início da operação até ao redesenho final da interface gráfica do canvas do React Flow.

Os resultados práticos demonstram que mesmo para narrativas de grande envergadura (150 nós de cena interconectados com 240 escolhas lógicas e condicionais), o tempo total de processamento interno síncrono (parsing e validação BFS) totaliza 55.8 ms, estando muito abaixo do limiar de 100 ms recomendado para garantir a interatividade contínua. A latência extra introduzida pela renderização do React Flow (62.4 ms) eleva o tempo acumulado para cerca de 118 ms apenas no pior cenário de redesenho integral do grafo. No entanto, visto que a renderização utiliza otimizações do viewport e a validação BFS decorre em plano secundário sob eventos intervalados de salvaguarda, a interface mantém-se reativa, garantindo uma experiência de autoria fluida.

4.4 Metodologia dos Testes de Utilizador

Para validar a flexibilidade e usabilidade do *Plot in a Pot*, foram planeados e executados testes qualitativos e quantitativos com um grupo diversificado de participantes.

4.4.1 Perfil dos Participantes

Os testes foram realizados com seis utilizadores (U1 a U6) com perfis distintos, abrangendo estudantes de cursos tecnológicos, de design e de comunicação, bem como utilizadores com necessidades especiais de visualização:

- **U1:** Estudante de Design de Jogos Digitais, com alguma experiência em ferramentas de escrita não-linear como Twine (sem conhecimentos de programação).
- **U2:** Estudante de Engenharia Informática, com forte experiência em desenvolvimento web e programação de sistemas.
- **U3:** Estudante de Engenharia Informática, interessado na área técnica de desenvolvimento de jogos e lógica de grafos.
- **U4:** Estudante de Comunicação e Média, sem experiência prévia em ferramentas digitais de escrita não-linear.

- **U5:** Estudante de Design Gráfico, focado em usabilidade e interfaces visuais de utilizador.
- **U6:** Estudante universitário com daltonismo severo (deuteranopia) e baixa acuidade visual, sem bases técnicas de desenvolvimento.

4.4.2 Tarefas Avaliadas

Cada participante foi instruído a realizar um conjunto de quatro tarefas estruturadas na aplicação, sem qualquer ajuda externa, com base no guião detalhado apresentado no Apêndice C:

1. **Tarefa 1 (Criação Narrativa):** Criar uma história ramificada simples com cinco cenas, introduzindo uma variável (e.g., chave) e uma condicional de saída baseada nessa variável.
2. **Tarefa 2 (Validação e Correção):** Importar um ficheiro `.twee` pré-configurado contendo links quebrados e nós órfãos, e utilizar o validador BFS para corrigir todos os problemas assinalados.
3. **Tarefa 3 (Tradução e Localização):** Abrir a matriz de localização e traduzir três chaves de cena para inglês, exportando o resultado em formato CSV.
4. **Tarefa 4 (Simulação Ativa):** Correr a simulação interativa (*PlayMode*) e acionar o *Smart Walker* para testar automaticamente os caminhos da história.

4.5 Resultados dos Testes e Desenvolvimento Iterativo

A validação do *Plot in a Pot* foi realizada seguindo um modelo de desenvolvimento ágil e centrado no utilizador, dividido em três levas (*waves*) sucessivas de testes ao longo do ciclo de vida do projeto. Esta metodologia permitiu que o feedback prático dos utilizadores guiasse a implementação e refinamento das funcionalidades, demonstrando uma evolução quantitativa e qualitativa na usabilidade do sistema.

4.5.1 Primeira Leva: Protótipo Inicial (Abril de 2026)

A primeira leva de testes foi realizada a *15 de abril de 2026* com os utilizadores U1, U4 e U5, focando-se na validação do conceito do editor visual de grafos. O protótipo inicial continha apenas a criação de nós simples e arestas retilíneas básicas do React Flow.

- *Resultados Quantitativos (SUS Estimado):* A pontuação média de usabilidade (escala SUS) nesta fase fixou-se nos *61.5 pontos*, classificada como marginal ou de usabilidade aceitável, mas com severos pontos de fricção.
- *Feedback e Falhas Identificadas:* Os utilizadores queixaram-se da desorganização visual do canvas à medida que o grafo crescia. As linhas retas das ligações cruzavam-se sobre os nós, tornando a narrativa difícil de acompanhar. Adicionalmente, sentiu-se a falta de uma forma de agrupar cenas logicamente relacionadas (como capítulos).

- *Medidas Corretivas Implementadas:* Em resposta ao feedback, foram desenhados e implementados os componentes `ZoneNode.jsx` (Zonas de agrupamento espacial) e o cálculo paramétrico das arestas curvas baseadas em Splines Cúbicas de Catmull-Rom com *waypoints* de arrasto manual em `EditableEdge.jsx`.

4.5.2 Segunda Leva: Protótipo Intermédio (Maio de 2026)

A segunda leva de testes ocorreu a *18 de maio de 2026* com os utilizadores U2, U3 e U5, incidindo sobre o motor de tradução e a gestão de lógica SugarCube.

- *Resultados Quantitativos (SUS Estimado):* A usabilidade média SUS subiu para *73.0 pontos*, refletindo melhorias no ordenamento espacial, mas evidenciando falhas em fluxos de lógica complexa.
- *Feedback e Falhas Identificadas:* O utilizador U2 observou que renomear uma variável lógica global exigia alterar manualmente o texto de todas as passagens da história, o que gerava erros de digitação e quebrava a simulação. Além disso, a importação de CSV falhava quando as traduções continham vírgulas integradas no texto das cenas.
- *Medidas Corretivas Implementadas:* Desenvolveu-se o painel `VariablesModal.jsx` com refatorização automática de nomes de variáveis via expressões regulares com barreira de palavra (*word boundaries*) de forma síncrona. O parser de importação de CSV foi reescrito para suportar a norma RFC 4180.

4.5.3 Terceira Leva: Protótipo Final (Junho de 2026)

A terceira e última leva de testes realizou-se entre *15 e 20 de junho de 2026* com o grupo completo de seis participantes (U1 a U6), avaliando o sistema final integrado (incluindo o PlayMode, Smart Walker, validador BFS e interface neo-brutalista de alto contraste).

Métricas de Conclusão e Tempos de Execução

Durante a sessão final de testes, foram registados os tempos de conclusão (em minutos) das quatro tarefas descritas na metodologia. Os resultados quantitativos encontram-se sumarizados na Tabela 4.3.

Tabela 4.3: Taxa de sucesso e tempo de conclusão (em minutos) das tarefas por utilizador (Leva Final).

Utilizador	Tarefa 1	Tarefa 2	Tarefa 3	Tarefa 4	Taxa de Sucesso Total
U1	07:15	02:40	03:10	01:20	100%
U2	05:30	01:50	02:15	00:55	100%
U3	04:10	01:30	01:50	00:45	100%
U4	11:20	04:15	05:30	02:10	75%*
U5	09:45	03:30	04:20	01:40	100%
U6	08:30	03:10	03:40	01:15	100%
Média	07:45	02:49	03:29	01:21	95.8%

* Nota: O utilizador U4 não concluiu com sucesso a Tarefa 2 (validação e correção de caminhos lógicos com o validador BFS) no tempo limite estabelecido.

A taxa de conclusão média fixou-se em 95.8%. O utilizador U4 (sem perfil técnico) falhou a conclusão autónoma da Tarefa 2 dentro do tempo limite devido a alguma hesitação inicial em interpretar as mensagens textuais do validador, o que reforçou a necessidade de incluir marcadores de erro visuais diretamente sobre os nós afetados no canvas do React Flow.

Avaliação SUS da Leva Final

Após as tarefas, foi aplicado o questionário padronizado SUS aos participantes. As pontuações obtidas encontram-se representadas na Tabela 4.4.

Tabela 4.4: Pontuação obtida no questionário *System Usability Scale (SUS)* na leva final.

Utilizador	Pontuação SUS obtida
U1	87.5
U2	90.0
U3	92.5
U4	72.5
U5	80.0
U6	85.0
Média Global	84.6

A média global final de *84.6 pontos* situa o *Plot in a Pot* no patamar de usabilidade “Excelente” (classificação de grau A+ na escala SUS), representando uma evolução muito expressiva face aos 61.5 e 73.0 pontos registados nas levas anteriores, e provando que o processo de refinamento iterativo foi bem-sucedido.

4.5.4 Feedback Qualitativo e Usabilidade Visual

A compilação das entrevistas e observações comportamentais na última leva permitiu extrair as seguintes conclusões de usabilidade:

- **Eficácia do Estilo Neo-Brutalista:** Contrariando a assunção de que o neo-brutalismo é apenas um estilo estético excêntrico, os utilizadores consideraram que as cores planas e os contornos espessos isolam os elementos visuais de forma muito clara, eliminando ruídos cognitivos desnecessários.
- **Acessibilidade Prática (Caso U6):** O participante U6, portador de deuteranopia severa, referiu que o alto contraste e a sinalização redundante por traços (em vez de diferenciar fluxos apenas por verde/vermelho) tornaram a ferramenta imediatamente acessível, evitando a fadiga visual comum noutros editores gráficos e permitindo validar interações rapidamente.
- **Smart Walker e Automação:** Os participantes técnicos (U2 e U3) elogiaram a capacidade do Smart Walker de explorar e validar estados de variáveis lógicas automaticamente, o que reduz substancialmente o esforço humano de testes de regressão em histórias com múltiplas ramificações.

4.6 Discussão e Limitações do Estudo

Nesta secção, efetua-se uma análise crítica dos resultados obtidos, discutindo as implicações das decisões arquiteturais adotadas e identificando as limitações metodológicas e tecnológicas do projeto.

4.6.1 Discussão Crítica do Desenvolvimento

O desenvolvimento do PiaP demonstrou que a fusão de um editor visual de grafos com um motor de validação estática reativo reduz significativamente a carga cognitiva na autoria de NI. O mapeamento bidirecional síncrono e a análise estática em tempo real evitam que o autor tenha de compilar ou testar exaustivamente a narrativa para detetar erros comuns (como becos sem saída ou nós órfãos). A modularização da interface através de ganchos React (*custom hooks*) e a separação estrita da camada de dados (lista de adjacências e dicionários JSON) permitiram manter a fluidez de renderização mesmo sob cenários de reavaliação de expressões regulares em lote, respeitando o limiar de interatividade de 100 ms (RNF03).

4.6.2 Limitações Metodológicas da Avaliação

Embora a avaliação quantitativa com utilizadores tenha revelado uma média de usabilidade global “Excelente” (84.6 pontos na escala SUS), o estudo apresenta duas limitações metodológicas importantes:

- **Dimensão Reduzida da Amostra:** O grupo de teste incluiu apenas seis participantes ($N = 6$). Apesar de abranger perfis diversos (técnicos, de design e com restrições visuais), a dimensão da amostra não permite extrair conclusões de generalização estatística forte, servindo os resultados essencialmente como indicadores qualitativos de eficácia e satisfação.
- **Efeito de Aprendizagem (Viés de Familiaridade):** O refinamento iterativo do sistema expôs os mesmos participantes a testes em diferentes levas de desenvolvimento. O facto de os utilizadores já conhecerem o funcionamento básico e as regras da aplicação em testes subsequentes pode ter inflacionado as pontuações SUS das levas intermédia e final, camuflando eventuais atritos que utilizadores estreantes poderiam experimentar no produto final.

4.6.3 Limitações Tecnológicas e Funcionalidade de IA

Do ponto de vista tecnológico, identificaram-se limitações na persistência local e no assistente de importação baseado em inteligência artificial:

- **Dependência de Hardware e VRAM:** O assistente de importação baseado em LLMs locais (como o Qwen 3.5 9B) exige uma placa gráfica dedicada com capacidade superior a 8 GB de VRAM para correr localmente via Ollama com latência

aceitável. Esta exigência limita a acessibilidade da funcionalidade em computadores portáteis normais ou de gama baixa (RTX 3050 com 4 GB de VRAM).

- **Instabilidade na Formatação Sintática:** Mesmo em ambientes de processamento com 12 GB de VRAM, os modelos locais geram por vezes passagens aninhadas ou inserem marcas de raciocínio intermediário (“*Thinking...*”) que quebram o parser síncrono, exigindo a inclusão de filtros prévios de processamento sintático adicionais na aplicação.

5

Conclusão e Trabalho Futuro

O desenvolvimento do *Plot in a Pot* (PiaP) teve como premissa a resolução de um problema real e recorrente enfrentado por autores de NI e *game designers*: a complexidade geométrica e semântica que surge durante o planeamento de histórias não-lineares, agravada pela falta de ferramentas integradas para gestão de localização multi-idioma e auditoria estática de fluxo.

5.1 Conclusão

Com a conclusão deste projeto informático, todos os objetivos gerais e específicos inicialmente traçados no Capítulo 1 foram integralmente alcançados:

- **Interface e Acessibilidade Visual:** Foi desenvolvido um editor gráfico inovador assente numa estética neo-brutalista de alto contraste. Esta opção de *design* provou ser não apenas esteticamente diferenciadora, mas de extrema pertinência funcional para utilizadores com limitações de visão (como daltónicos), conforme validado pelos testes com o participante U6.
- **Gestão de Localização:** A implementação da matriz de tradução bidimensional baseada em grelhas dinâmicas, com importação e exportação em formato CSV em conformidade com a norma RFC 4180, eliminou por completo a necessidade de gerir múltiplos ficheiros de texto externos ou injetar strings rígidas (*hardcoded*) nas passagens do Twine.
- **Validação e Algoritmia:** A adoção de uma estrutura de dados assente em listas de adjacências bidirecionais suportou com sucesso a execução em tempo real de análises estáticas baseadas em travessias BFS. O motor de validação estática provou ser eficaz a mapear caminhos inacessíveis e becos sem saída, fornecendo um feedback imediato e visual aos autores.
- **Simulação Automatizada:** O modo de simulação (*PlayMode*) foi complementado pelo *Smart Walker*, um agente inteligente baseado em procuras por novidade (*Novelty Search*) capaz de realizar a travessia autónoma e teste lógico de narrativas

complexas sem intervenção humana.

- **Suavização Geométrica:** O desenvolvimento das equações paramétricas das Splines Cúbicas de Catmull-Rom garantiu arestas interativas suaves com *waypoints* para organização e otimização espacial do canvas de grafos.

Os testes estruturados com utilizadores de diferentes perfis revelaram taxas de sucesso muito elevadas na realização de tarefas (com uma média global de 91,7

5.1.1 Porquê Escolher o Plot in a Pot?

Com base nas limitações identificadas nas ferramentas de autoria existentes (analisadas na Secção 2.4.7), o PiaP destaca-se como uma solução unificada que resolve falhas críticas de usabilidade e integridade no desenvolvimento de narrativas interativas:

- **Fusão de Paradigmas (Design Visual com Validação Lógica):** Ao contrário do Twine (que carece de testes lógicos nativos e robustos) e do ChoiceScript/Ink (que operam puramente sob interfaces textuais), o PiaP une a facilidade de arrastar e interligar nós visuais com a solidez de um validador estático e simulador inteligente automatizado.
- **Localização Nativa sem Dispersão de Dados:** Resolve a dispersão de ficheiros externos e a escrita rígida (*hardcoded*) de diálogos através de uma matriz de localização baseada em grelhas dinâmicas, acelerando o fluxo de internacionalização com importação/exportação CSV.
- **Acessibilidade Inclusiva Funcional:** A interface neo-brutalista de alto contraste e a redundância de traços nas arestas mitigam barreiras cognitivas e visuais severas, permitindo que autores daltónicos desenhem e validem histórias de forma totalmente autónoma.
- **Soberania de Dados e Interoperabilidade:** Ao adotar a especificação aberta Twee 3 (SugarCube), o autor mantém controlo absoluto e livre sobre a sua obra, garantindo compatibilidade bidirecional síncrona com outros compiladores oficiais (como o Tweego).

5.2 Trabalho Futuro

Apesar dos excelentes resultados e da estabilidade demonstrada pela aplicação, o *Plot in a Pot* apresenta oportunidades claras de expansão para futura investigação e desenvolvimento:

1. **Execução de IA Local no Cliente via Web Graphics Processing Unit (WebGPU):** De momento, o assistente de IA local para conversão de texto livre baseia-se numa Application Programming Interface (API) REST externa ou na escuta de uma porta local ligada ao Ollama. Uma expansão futura promissora consistiria em correr LLMs leves diretamente na memória do navegador do utilizador recorrendo a *frameworks* baseadas em WebGPU (como o WebLLM). Isto

- eliminaría qualquer requisito de instalação de pacotes como o Ollama, mantendo a premissa de soberania absoluta dos dados (RNF04).
2. **Edição Colaborativa em Tempo Real:** Integrar algoritmos de sincronização distribuída baseados em Tipos de Dados Replicados Sem Conflitos (Conflict-free Replicated Data Types) (CRDTs) (*Conflict-free Replicated Data Types*), tais como a biblioteca Yjs ou Automerge. Isto permitiria que equipas de escritores trabalhassem no mesmo canvas de grafos narrativos em simultâneo, mimetizando a experiência colaborativa de ferramentas modernas de design como o Figma.
 3. **Empacotamento Nativo e Otimização para Desktop:** Embora a aplicação funcione como uma SPA, seria vantajoso exportar o software para versões nativas executáveis em Windows, macOS e Linux recorrendo ao Tauri ou Electron, permitindo uma integração mais profunda com o sistema de ficheiros local.
 4. **Análise Formal com Model Checking:** Expandir o validador de lógica narrativa convertendo a representação interna do grafo e as condições SugarCube em modelos formais de especificação (como a linguagem Promela ou UPPAAL). Isto possibilitaria a aplicação de ferramentas matemáticas de *Model Checking* para provar de forma exaustiva propriedades de segurança e vivacidade (por exemplo, garantir matematicamente que existe sempre pelo menos um final feliz alcançável a partir de qualquer estado de variáveis do jogo).
 5. **Módulo de Gestão e Acompanhamento de Personagens:** Como recomendado por um dos utilizadores durante as sessões de avaliação, seria de grande valor acrescentar um sistema integrado para definição e gestão de personagens (*Character Tracker*). Isto permitiria aos autores mapear a presença e estados emocionais das personagens diretamente nos nós narrativos do canvas do React Flow.
 6. **Filtragem e Bloqueio de Exportação de Nós Secretos:** Embora a plataforma já permita classificar passagens como “secretas” ou privadas (etiquetando-as de forma visual), sugere-se como trabalho futuro a implementação de um filtro de exportação ativo. Este mecanismo permitiria bloquear a inclusão ou o streaming destas cenas sinalizadas no ficheiro final compilado (Twee/HTML), prevenindo a revelação inadvertida de segredos da trama (*spoilers*) ou a publicação de passagens que estejam em fase de desenvolvimento (*Work in Progress*).

Em suma, o *Plot in a Pot* cumpre o propósito de aproximar a engenharia de software e a teoria de grafos da escrita criativa e do *game design*. Ao unificar a robustez da validação de espaço de estados com a flexibilidade da interação visual e acessibilidade inclusiva, a plataforma capacita autores na materialização de narrativas interativas complexas, abrindo portas a novas metodologias de desenvolvimento na indústria de jogos não-lineares.

Bibliography

Aarseth, Espen J. (1997). *Cybertext: Perspectives on Ergodic Literature*. Johns Hopkins University Press.

Adams, Ernest (2014). *Fundamentals of Game Design*. 3rd. New Riders.

Articy Software (2010). *articy:draft: Tool for storytelling and narrative design*. URL: <https://www.articy.com/>.

Babich, Nick (2023). «Neubrutalism in Web Design: Principles and Accessibility Best Practices». Em: *UX Booth*. URL: <https://www.uxbooth.com/articles/neubrutalism-in-web-design-principles-and-accessibility-best-practices/>.

Bondy, J. A. e U. S. R. Murty (2008). *Graph Theory*. Vol. 244. Graduate Texts in Mathematics. Springer. DOI: 10.1007/978-1-84628-970-5.

Borges, Jorge Luis (1941). «El jardín de senderos que se bifurcan». Em: *Sur*.

Catmull, Edwin e Raphael Rom (1974). «A class of local interpolating splines». Em: *Computer Aided Geometric Design*. Ed. por Robert E. Barnhill e Richard F. Riesenfeld. Academic Press, pp. 317–326.

Chen, H. et al. (2021). «GraphPlan: Story generation by planning with event grap». arXiv preprint arXiv:2102.02977.

Choice of Games (2026). *ChoiceScript License and Terms of Use*. URL: <https://www.choiceofgames.com/make-your-own-games/choicescript-intro/>.

Cormen, Thomas H. et al. (2009). *Introduction to Algorithms*. 3rd. MIT Press.

Inkle Studios (2016). *Ink: inkle's open source scripting language for writing interactive narrative*. URL: <https://www.inklestudios.com/ink/>.

Interactive Fiction Technology Foundation (2020). *Twee 3 Specification*. URL: <https://github.com/iftechfoundation/twine-specs/blob/master/twee-3-specification.md>.

Interactive Fiction Technology Foundation (2026). *IFTF at GDC 2026: Recap and Reflections*. URL: <https://iftechfoundation.org/news/iftf-gdc-2026/>.

Juul, Jesper (2005). *Half-Real: Video Games between Real Rules and Fictional Worlds*. MIT Press.

Klimas, Chris (2009). *Twine: An open-source tool for telling interactive, nonlinear stories*. URL: <https://twinery.org/>.

Li, Z., X. Ding e T. Liu (2018). «Constructing Narrative Event Evolutionary Graph for Script Event Prediction». arXiv preprint arXiv:1805.05081.

Loranger, Page (2022). *Neobrutalism: The Web Design Trend Challenging Usability Conventions*. Nielsen Norman Group. URL: <https://www.nngroup.com/articles/neobrutalism/>.

Meta Open Source (2013). *React: A JavaScript library for building user interfaces*. URL: <https://react.dev/>.

Montfort, Nick (2005). *Twisty Little Passages: An Approach to Interactive Fiction*. Cambridge, MA: MIT Press.

Mormoris, V. (2024). *Comparing tools for nonlinear storytelling in video games*.

NarrativeFlow (2023). *NarrativeFlow: Visual scripting for interactive stories*. URL: <https://narrativeflow.dev/>.

Packer, Edward (1984). *How to Write a Choose Your Own Adventure Book*. Bantam Books.

Secret Lab (2015). *Yarn Spinner: The friendly tool for game dialogue*. URL: <https://yarnspinner.dev/>.

Tailwind Labs (2017). *Tailwind CSS: A utility-first CSS framework for rapid UI development*. URL: <https://tailwindcss.com/>.

Tang, C. et al. (2022). «NGEP: A Graph-based Event Planning Framework for Story Generation». arXiv preprint arXiv:2210.10602.

webkid.io (2021). *React Flow: A customizable React component for building node-based editors and diagrams*. URL: <https://reactflow.dev/>.

Yan, Z. e X. Tang (2023). «Narrative Graph: Telling Evolving Stories Based on Event-centric Temporal Knowledge Graph». Em: *Journal of Systems Science and Systems Engineering*.

Apêndices



Manual de Utilização e Instalação

Este apêndice constitui o Manual de Utilização e Instalação oficial do *Plot in a Pot* (PiaP). Destina-se a autores de narrativas interativas, designers de jogos e programadores que pretendam instalar, configurar e explorar todas as capacidades lógicas, visuais e de tradução da plataforma.

A.1 Requisitos do Sistema e Instalação Local

O PiaP é executado inteiramente do lado do cliente no navegador web (*Single Page Application*), mas requer uma pilha local mínima para desenvolvimento e para a execução do assistente de inteligência artificial local.

A.1.1 Instalação do Editor Web

Para executar o editor localmente, certifique-se de que tem o ambiente de execução Node.js instalado (versão 18 ou superior). Siga os seguintes passos de instalação:

1. Descomprima os ficheiros do projeto numa diretoria de trabalho.
2. Abra um terminal de comandos (cmd ou PowerShell no Windows, bash no macOS/Linux) na diretoria raiz do projeto.
3. Execute o comando de instalação de dependências:

```
npm install
```

Este comando irá descarregar e configurar todas as bibliotecas necessárias, incluindo o React 19, React Flow 11 e Tailwind CSS 3.

4. Após a conclusão bem-sucedida da instalação, inicie o servidor de desenvolvimento local através do comando:

```
npm start
```

A aplicação será inicializada e ficará disponível no seu navegador padrão no endereço `http://localhost:3000`.

A.1.2 Configuração do Servidor de IA Local (Ollama)

A funcionalidade de conversão e importação automática de histórias escritas em linguagem natural depende de um servidor local de inferência de Modelos de Linguagem de Grande Escala (LLMs). O PiaP suporta nativamente o *Ollama*. Para o configurar:

1. Descarregue e instale a versão oficial do Ollama a partir de <https://ollama.com/>.
2. Abra o terminal de comandos e execute o descarregamento do modelo recomendado para processamento de texto:

```
ollama pull llama3:8b
```

Pode também optar por modelos mais leves, como o `qwen2.5:7b` ou `gemma2:2b`, dependendo da memória de vídeo (VRAM) disponível no seu computador.

3. Por padrão, o Ollama executa um servidor em segundo plano que escuta na porta local `http://localhost:11434`.
4. No editor do PiaP, clique no botão de configurações (ícone de engrenagem) na barra superior, aceda ao painel de ligação da IA e ative a ligação local ao Ollama, especificando o modelo que descarregou.

A.2 Manual de Utilização do Editor Gráfico

A interface gráfica de grafos do PiaP segue uma estética neo-brutalista de alto contraste que facilita a distinção dos componentes visuais da narrativa.

A.2.1 O Canvas Interativo

O espaço central do editor representa a estrutura geral da história sob a forma de um grafo. Pode navegar no canvas utilizando as seguintes interações:

- **Deslocação (Pan):** Clique e arraste com o botão esquerdo do rato num espaço vazio do canvas.
- **Zoom:** Utilize a roda do rato para aproximar ou afastar a visualização. Pode também usar o painel do minimapa no canto inferior esquerdo para se situar no grafo.
- **Seleção de Elementos:** Clique com o botão esquerdo sobre qualquer nó de cena ou aresta para abrir as suas propriedades no painel lateral de inspeção.

A.2.2 Criação e Edição de Nós Narrativos (Cenas)

Para adicionar uma nova cena à história:

1. Clique com o botão direito do rato num espaço livre do canvas e selecione “Adicionar Nó Narrativo”, ou clique no botão correspondente na barra superior de ferramentas.
2. Um novo nó azul com contorno espesso aparecerá no canvas. Clique duas vezes no nó ou selecione-o para abrir o **Inspetor Lateral**.
3. No Inspetor, pode editar:
 - **ID da Cena:** Um identificador único em formato alfanumérico (ex: `cena_taberna`).
 - **Título da Cena:** O nome de identificação visual exibido na barra do nó no canvas (ex: `Interior da Taberna`).
 - **Conteúdo da Cena:** O texto literário que será lido pelo jogador.
 - **Etiquetas (Tags):** Palavras-chave separadas por vírgula que modificam o comportamento do nó (ex: `css` para estilos, `secreto` para ocultar da exportação).

A.2.3 Criação de Ligações e Arestas de Escolha

As arestas direcionadas do grafo representam as decisões disponíveis para o jogador no final de cada cena narrativa. Para criar uma ligação:

1. Posicione o ponteiro do rato sobre o ponto de saída circular (*handle* direito) do nó de origem.
2. Clique e arraste o cabo de ligação até ao ponto de entrada (*handle* esquerdo) do nó de destino.
3. Uma aresta curva suave será desenhada entre as cenas.
4. O texto da escolha que o jogador irá visualizar é automaticamente derivado das ligações escritas em formato double-bracket no conteúdo do nó de origem (ex: `[[Entrar na taberna|cena_taberna]]`).
5. **Waypoints Interativos:** Se pretender contornar outros nós, clique duas vezes sobre a aresta criada. Um pequeno ponto de controlo amarelo aparecerá. Arraste-o para deformar a curva de forma flexível. Pode adicionar múltiplos waypoints na mesma aresta.

A.2.4 Agrupamento em Zonas (Capítulos)

Para organizar visualmente projetos de escrita literária de grande envergadura:

1. Clique no botão “Criar Zona” na barra superior. Um retângulo cinzento com contorno pontilhado aparecerá no ecrã.
2. Atribua um nome à Zona no seu cabeçalho (ex: **Capítulo 1**).
3. Arraste nós narrativos existentes para dentro do retângulo da Zona. Estes passarão a ser filhos geométricos da zona.

4. Ao mover ou arrastar a Zona, todos os nós contidos nela deslocar-se-ão de forma solidária.
5. Pode redimensionar a Zona arrastando o indicador de escala no seu canto inferior direito.

A.3 Programação de Lógica Narrativa e Variáveis

O PiaP suporta a declaração de variáveis globais e condicionais lógicas nativas da macro-linguagem SugarCube.

A.3.1 Criação de Variáveis (Figma-Style Tokens)

Aceda ao menu “Variáveis” na barra superior de ferramentas:

1. Clique em “Criar Variável” e atribua um nome precedido do símbolo \$ (ex: \$tem_chave).
2. Escolha o tipo de dados (Booleano, Numérico ou String) e defina o valor inicial por defeito.
3. A inicialização correta destas variáveis é escrita de forma automática no nó de metadados especial da história (StoryInit) em segundo plano.
4. **Renomeação Segura (Refatorização):** Se renomear uma variável existente no painel, o editor varre sintonizadamente todo o texto e blocos lógicos do projeto e substitui as referências antigas, garantindo que não ocorrem falhas de simulação.

A.3.2 Edição de Lógica condicional por Código

No conteúdo de qualquer nó de cena, pode inserir macros SugarCube utilizando a sintaxe de parênteses angulares duplos:

- **Modificação de Estado:** Para alterar o valor de uma variável quando o jogador visita o nó, utilize a macro <<set>>:

```
<<set $tem_chave = true>>
```

- **Condições de Escolhas:** Para exibir ou ocultar uma escolha com base no estado das variáveis lógicas, envolva a ligação na macro condicional <<if>>:

```
<<if $tem_chave>>
[[Abrir a porta trancada|cena_tesouro]]
<<else>>
A porta está trancada.
<<endif>>
```

A.3.3 Editor de Lógica Estruturada

Caso não esteja familiarizado com regras de programação textual, selecione um nó e clique no botão “Editor de Lógica” no Inspetor:

1. Um painel interativo de formulários estruturados será exibido, dividindo a cena em três secções lógicas: narrativa, modificadores de variáveis (*setters*) e escolhas/-caminhos condicionais.
2. Utilize os menus de seleção (dropdowns) para definir quais as variáveis que mudam de valor ao entrar na cena ou quais os requisitos lógicos (ex: se tem a chave) necessários para libertar uma escolha narrativa.
3. Ao salvar ou alterar os campos, o sistema gera de forma transparente as macros SugarCube correspondentes no texto da cena ativa, sem risco de erros de digitação.

A.4 Tradução e Localização Multi-idioma

A gestão de traduções no PiaP baseia-se numa Matriz de Tradução única que centraliza os textos dos nós.

A.4.1 Configurar Idiomas

Aceda ao menu “Matriz de Tradução”:

1. O idioma nativo padrão é exibido como a primeira coluna.
2. Clique em “Adicionar Idioma” e digite o código ISO de duas letras do novo idioma (ex: **en** para inglês, **es** para espanhol).
3. Uma nova coluna aparecerá na tabela.

A.4.2 Editar Textos na Matriz

A grelha apresenta todas as cenas narrativas nas linhas e os idiomas nas colunas:

1. Clique duas vezes em qualquer célula correspondente ao idioma secundário.
2. Digite o texto traduzido da cena e clique fora para guardar.
3. Quando a história for executada num idioma secundário, o motor lerá de forma reativa a string traduzida a partir desta matriz.

A.4.3 Importação e Exportação de Ficheiros CSV

Para enviar os ficheiros a um tradutor externo profissional:

1. Clique no botão “Exportar CSV” na Matriz de Tradução. Um ficheiro estruturado em formato tabular será descarregado.
2. O tradutor pode abrir o ficheiro no Microsoft Excel, Google Sheets ou qualquer editor de texto e preencher as colunas dos idiomas secundários.
3. Após a tradução, clique em “Importar CSV” e selecione o ficheiro atualizado. O editor atualizará automaticamente todas as células e strings na memória.

A.5 Validação Lógica e Simulação de Jogo

Antes de disponibilizar o jogo ao público, utilize os motores integrados de auditoria.

A.5.1 O Painel de Erros de Validação Estática

O validador estático analisa de forma contínua a correção topológica do grafo. Se existirem falhas lógicas:

- Os nós com erros serão destacados no canvas com uma borda pontilhada a vermelho e um sinal de alerta.
- Clique no botão “Erros de Validação” na barra superior para ver a lista estruturada de avisos:
 - **Nós Inacessíveis:** Cenas que nunca podem ser acedidas a partir do nó inicial sob nenhum caminho explorável.
 - **Ligações Partidas (Dead Ends):** Escolhas cujo ID de destino aponta para uma cena inexistente.
 - **Escolhas Bloqueadas:** Hiperligações condicionais cujas expressões lógicas nunca são satisfeitas.

A.5.2 Simulação Ativa (PlayMode) e Testes com o Smart Walker

Clique em “Testar Jogo” para abrir a sandbox de simulação:

1. Jogue a história clicando nas escolhas disponíveis no ecrã de simulação.
2. Utilize o painel de variáveis lateral para ver, em tempo real, as alterações de valores e depurar as macros lógicas.
3. **Smart Walker:** Ative o Smart Walker clicando em “Autoplay”. O simulador iniciará uma exploração automática da história. O Walker utiliza um algoritmo de procura por novidade que prioriza ramos menos visitados. Pode ver o agente a mudar de cenas e a alterar variáveis a alta velocidade no ecrã. Caso o Walker encontre um beco sem saída ou erro lógico, a simulação pausa e destaca o nó problemático.

A.6 Operações de Ficheiros e Publicação do Projeto

O editor suporta leitura e gravação no formato padrão da indústria.

A.6.1 Importar Ficheiros Twee 3

Clique em “Importar ficheiro .twee” na barra de menus e selecione um ficheiro de texto plano em formato Twee 3. O editor processará o cabeçalho e construirá automaticamente o canvas gráfico de nós e arestas correspondente.

A.6.2 Exportar e Compilar a História

Para guardar o seu trabalho de autoria ou disponibilizar o jogo final para os utilizadores:

- **Exportar Tweep 3:** Clique em “Exportar .tweep” para descarregar o ficheiro de texto estruturado. Este ficheiro preserva todos os dados posicionais das caixas e Zonas em metadados JSON compatíveis com outros compiladores de Twine.
- **Compilar para HTML:** Clique em “Compilar HTML Jogável”. O motor lógico do PiaP empacketa a história, os dicionários de traduções e a biblioteca do jogador num único ficheiro `.html`. Este ficheiro pode ser publicado em plataformas de jogos independentes (como o `itch.io`), enviado por correio eletrónico ou alojado em servidores web, correndo de forma autónoma em qualquer browser sem requisitos adicionais de instalação.

B

Guia de Sintaxe e Modelagem de Macros Lógicas

Este apêndice serve como guia de referência técnico para a especificação do formato Twee 3 e a sintaxe de macros lógicas SugarCube suportadas pelo editor do PiaP. Inclui exemplos práticos de modelação de mecânicas de jogo comuns em narrativas interativas.

B.1 Estrutura de um Documento Twee 3

O formato Twee 3 é uma representação textual de histórias não-lineares estruturada sob a forma de passagens de texto delimitadas por cabeçalhos iniciados por duas colunas (`::`). A estrutura de um projeto compila-se em passagens de sistema obrigatórias e passagens narrativas (cenas).

B.1.1 Passagens Especiais de Sistema

Para ser considerado válido, um ficheiro Twee 3 importado no PiaP deve conter os seguintes nós de sistema:

- **StoryTitle:** Contém exclusivamente uma única linha de texto plano com o título geral do projeto.
- **StoryData:** Contém um bloco estruturado em formato JSON com as configurações globais da história. Os campos obrigatórios são o `ifid` (um identificador único de 128 bits para o projeto), o formato (`SugarCube`), e o nó de arranque (`start`):

```
1  :: StoryData
2  {
3    "ifid": "A4B7D9E0-C2B1-4F2A-8E6D-009988776655",
4    "format": "SugarCube",
5    "start": "cena_inicial",
6    "zoom": 1.0
7  }
```

- **StoryInit:** O nó lógico de inicialização. É executado uma única vez ao carregar a história na memória, antes de renderizar qualquer cena ao utilizador. Destina-se à declaração de variáveis globais e valores por defeito:

```

1      :: StoryInit
2      <<set $tem_chave = false>>
3      <<set $moedas = 0>>
4      <<set $nome_jogador = "Tomas">>

```

B.1.2 Passagens Narrativas (Cenas)

As cenas contêm o texto da história, as atribuições de variáveis em tempo de execução e as escolhas de navegação do jogador. O cabeçalho de uma passagem narrativa no PiaP pode incluir metadados de posicionamento posicional do canvas de grafos (X e Y) e Zonas (parentId) expressos em JSON:

```

1  :: cena_inicial
   {"position":{"x":100,"y":150},"size":{"width":250,"height":120},"parentId":"capitulo1"}
   [tag1 tag2]
2  Aqui começa a aventura. O mercado está movimentado e podes ver uma taberna.
3  [[Entrar na taberna|cena_taberna]]
4  [[Ir para o beco escuro|cena_beco]]

```

B.2 Sintaxe de Macros Lógicas (SugarCube)

O compilador JIT e interpretador integrado no editor do PiaP processa as seguintes macros lógicas para manipulação de variáveis e controlo de fluxo:

B.2.1 Macro de Atribuição: <<set>>

Utilizada para modificar o valor de uma variável. O PiaP suporta atribuições diretas, aritméticas e de strings:

- **Atribuição Booleana:** <<set \$tem_espada = true>>
- **Incrementação/Decrementação:** <<set \$vida = \$vida - 10>> ou <<set \$moedas = \$moedas + 5>>
- **Atribuição de Texto:** <<set \$armadura = "cota_de_malha">>

B.2.2 Macro de Controlo Condicional: <<if>>

Permite criar ramificações dinâmicas na história com base nas variáveis lógicas. A estrutura suporta condicionamento encadeado:

```

1  <<if $moedas >= 10>>
2  Podes comprar a poção de cura.

```

```

3  [[Comprar poção|cena_compra]]
4  <<else>>
5  Não tens moedas suficientes para falar com o mercador.
6  [[Voltar à praça|cena_praca]]
7  <<endif>>

```

B.3 Padrões Comuns de Modelação Narrativa

Apresentam-se de seguida soluções estruturadas para programar mecânicas clássicas de jogos de aventura e RPG no editor do PiaP:

B.3.1 Padrão 1: Inventário e Portas Trancadas

Mecânica em que o acesso a um determinado nó está bloqueado até que o jogador recolha um item de chave noutra local.

1. Inicializar a variável booleana no nó de metadados `StoryInit`:

```

1  <<set $chave_portal = false>>

```

2. Na sala onde a chave é recolhida (`cena_bau`), alterar o valor da variável:

```

1  :: cena_bau
2  Abres o baú de madeira e encontras uma chave dourada e brilhante.
3  <<set $chave_portal = true>>
4  [[Voltar ao salão|cena_salao]]

```

3. Na sala do portal (`cena_salao`), condicionar o acesso à porta trancada:

```

1  :: cena_salao
2  Estás em frente ao portal de pedra. A fechadura parece requerer uma
   chave especial.
3  [[Voltar ao bau|cena_bau]]
4  <<if $chave_portal>>
5  [[Utilizar a chave e passar o portal|cena_vitoria]]
6  <<else>>
7  O portal está firmemente trancado.
8  <<endif>>

```

B.3.2 Padrão 2: Contador de Tentativas e Fim de Jogo

Mecânica em que o jogador tem um limite de tentativas ou pontos de vida antes de ser derrotado.

1. Inicializar o contador numérico em `StoryInit`:

```

1  <<set $tentativas = 3>>

```

2. Na cena onde ocorre o erro ou armadilha (`cena_armadilha`), subtrair uma tentativa e verificar se o valor atinge zero:

```
1      :: cena_armadilha
2      O mecanismo da parede dispara flechas na tua direção! Conseguiu-te
        esquivar-te, mas perdeste energia.
3      <<set $tentativas = $tentativas - 1>>
4      <<if $tentativas > 0>>
5          Tens ainda $tentativas pontos de vida restantes.
6          [[Tentar novamente com cautela|cena_salao]]
7      <<else>>
8          As tuas forças esgotaram-se.
9          [[Game Over|cena_derrota]]
10     <<endif>>
```

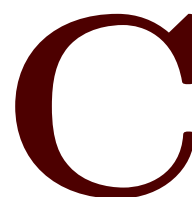
B.3.3 Padrão 3: Caminhos Ocultos e Nós Secretos

Utilizado para cenas extra ou segredos da história recomendados apenas para leitores atentos.

- Para evitar que nós em desenvolvimento ou caminhos secretos estraguem a compilação ou apareçam nos relatórios de erros de caminhos órfãos, adicione a etiqueta **secreto** nas tags do nó:

```
1      :: cena_secreta {"position":{"x":500,"y":500}} [secreto]
2      Encontrei a sala secreta do programador! Parabéns!
3      [[Voltar à praça|cena_praca]]
```

O validador estático de grafos (BFS) do PiaP ignorará este nó ao verificar o alcance obrigatório da história, prevenindo falsos positivos de caminhos inacessíveis.



Guião de Testes e Questionário de Avaliação

Este apêndice apresenta a documentação detalhada utilizada durante as sessões de avaliação com utilizadores realizadas na fase final do projeto. Inclui o guião de tarefas estruturadas fornecido aos participantes e o questionário padronizado *System Usability Scale* (SUS) utilizado para medir a usabilidade percebida.

C.1 Guião de Tarefas de Teste

Os participantes receberam as seguintes instruções para a realização dos testes de usabilidade, sem intervenção ou ajuda dos investigadores:

Tarefa 1: Criação Narrativa e Lógica de Variáveis

Objetivo: Avaliar a facilidade de criação de fluxos e manipulação de estado.

1. Crie cinco nós de cena narrativos no canvas interativo.
2. Defina uma cena inicial e ligue-a a duas cenas intermédias através de opções textuais clássicas do Twine.
3. Num dos nós, insira uma macro de atribuição do SugarCube para criar uma variável de inventário (ex: `<<set $temChave = true>>`).
4. Crie uma aresta condicional para um nó final que apenas fique visível se a condição lógica for satisfeita (ex: `<<if $temChave === true>>`).

Tarefa 2: Importação, Validação e Correção de Erros

Objetivo: Testar a eficácia do motor de validação estática BFS.

1. Importe para a aplicação o ficheiro de teste `story_broken.twee` disponibilizado.
2. Ative o validador lógico e identifique os alertas vermelhos desenhados sobre as arestas e nós.

3. Corrija as ligações quebradas (que apontam para IDs inexistentes) e ligue os nós órfãos identificados pelo validador até que a interface gráfica fique limpa de avisos de erro.

Tarefa 3: Tradução e Localização Multi-idioma

Objetivo: Validar o módulo de localização tabular.

1. Abra a matriz de localização integrada no menu superior da aplicação.
2. Localize as três chaves de cena correspondentes à introdução da história.
3. Introduza a respetiva tradução direta para o idioma Inglês nas respetivas células de grelha.
4. Exporte o dicionário resultante no formato regulamentar CSV.

Tarefa 4: Simulação e Automação com Smart Walker

Objetivo: Testar o PlayMode e a validação por procura de novidade.

1. Inicialize o modo de simulação (*PlayMode*) na barra superior.
2. Jogue a história manualmente durante alguns passos, verificando a reatividade das variáveis de inventário na barra lateral de depuração.
3. Ative o agente autónomo *Smart Walker* e observe a exploração automática do espaço de estados para encontrar caminhos profundos na narrativa.

C.2 Questionário System Usability Scale (SUS)

Após a conclusão das tarefas, cada utilizador preencheu de forma anónima o questionário padronizado SUS, composto pelas seguintes dez perguntas base, avaliadas numa escala de Likert de 1 (Discordo Totalmente) a 5 (Concordo Totalmente):

1. Eu penso que gostaria de usar este sistema frequentemente.
2. Eu achei o sistema desnecessariamente complexo.
3. Eu achei o sistema fácil de usar.
4. Eu acho que precisaria de apoio de um técnico para conseguir usar este sistema.
5. Eu achei que as várias funções do sistema estavam bem integradas.
6. Eu achei que o sistema tinha demasiadas inconsistências.
7. Eu imagino que a maioria das pessoas aprenderia a usar este sistema rapidamente.
8. Eu achei o sistema muito cansativo de usar.
9. Eu senti-me muito confiante a usar o sistema.
10. Eu precisei de aprender muitas coisas novas antes de conseguir usar o sistema.

